台灣自編大學教科書

地圖建構與同步定位 (SLAM)

Simultaneous Localization and Mapping



大學教科書

第一篇

第 1 ～ 3 章

SLAM 與自主導航概論

李世淵 (機器人叫獸)

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Part I Introduction to SLAM and Autonomous Navigation

Chapter 1

What Is Simultaneous Localization and Mapping?

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Before turning to the material itself, we explain the role this chapter plays in the book as a whole and how it connects to the chapters that follow. This "map" appears at the start of every chapter so that you always know where you are and where you are going next—which is precisely the localization problem this book is about, transposed into the space of knowledge.

Figure 1-0 How Chapter 1 Connects to the Rest of the Book

[This chapter is responsible for] establishing the central problem of SLAM: how a robot can know where it is and draw the shape of its environment at the same time.

[Where it leads]

The six sensors of Section 1.3 → examined one by one in Part II (Chapters 4–9): indoor positioning, satellites, geomagnetic field, IMU, LiDAR, camera

The chicken-and-egg problem of Section 1.2 → solved by probabilistic filtering in Chapter 11 and by graph optimization in Chapter 12

The four applications of Section 1.4 → Part IV robotic vacuums, Part V autonomous inspection, Part VI AR / VR, Part VII UAVs

The map representations of Section 1.5 → occupancy grids in Chapter 13, point clouds and octrees in Chapter 17, semantic maps in Chapter 32

The front end and back end of Section 1.6 → formally defined in Chapter 12 and realized in actual systems in Chapters 15, 18, and 22

The technical challenges of Section 1.7 → the thread of "common failures" running through the whole book

[Prerequisites] None. This chapter requires no mathematics and no programming background—only curiosity.

Learning Objectives

This chapter is the starting point of the book. Its purpose is to give you an overall picture of Simultaneous Localization and Mapping (SLAM). After completing this chapter, you should be able to:

1. State the basic definition of SLAM, and explain in your own words what problem localization solves and what problem mapping solves.
2. Explain why localization and mapping are preconditions for each other, forming a chicken-and-egg problem of circular dependency, and how SLAM estimates both at the same time.
3. List the sensors commonly used on robots (wheel odometry, IMU, camera, LiDAR, radio, geomagnetic field) and state the physical quantity each one measures, together with its strengths and weaknesses.
4. Give examples of the role SLAM plays in autonomous driving, unmanned aerial vehicles, robotic vacuum cleaners, and AR / VR.
5. Distinguish among the output forms of a SLAM system: trajectory, 2D occupancy grid map, 3D point cloud, semantic map, and topological map.
6. Complete the hands-on activity for this chapter and experience firsthand the difference between "the world as the sensor sees it" and the actual map.

Core Concepts

Term (Chinese)	Term (English)	Explanation
同步定位與建圖	SLAM	The technique by which a robot builds a map of an unknown environment while estimating its own position within it.
定位	Localization	Estimating the position and orientation (pose)

Term (Chinese)	Term (English)	Explanation
		of the robot in a map or in world coordinates.
建圖	Mapping	Building a geometric or semantic representation of the environment from sensor data.
位姿	Pose	Position (x, y, z) together with orientation; in 2D commonly written (x, y, θ).
里程計	Odometry	Relative displacement accumulated from wheel encoders or sensors; drifts over time.
慣性量測單元	IMU	Composed of an accelerometer and a gyroscope; measures acceleration and angular rate.
光達	LiDAR	A sensor that measures range by laser time of flight, producing scan lines or point clouds.
閉環偵測	Loop Closure	Recognizing "I have been here before" on returning to a previously visited place, used to correct accumulated error.
佔據格地圖	Occupancy Grid Map	A 2D map that divides space into small cells and marks each as occupied, free, or unknown.
點雲	Point Cloud	A spatial representation composed of a large number of three-dimensional points, usually produced by a 3D LiDAR or a depth camera.
語意地圖	Semantic Map	A map that, beyond geometric shape, annotates meanings such as door, table, and corridor.
拓樸地圖	Topological Map	A map describing places and the passages between them as nodes and edges, similar to a metro route diagram.

1.1 The Basic Definition of SLAM: Localization + Mapping

Picture the following situation. You are blindfolded, led into a building you have never visited, and then the blindfold is removed. You have no floor plan in hand, and there are no signs on the walls. What would you do? Most people would first look around and make mental notes—"there is a door on the left, a long corridor ahead, a staircase on the right"—and then start walking. As you walk, a picture of the building gradually forms in your mind, and at the same time you always know roughly that "I am now about ten meters ahead and to the right of that door."

This seemingly natural process is one of the most classic and most difficult problems in robotics: Simultaneous Localization and Mapping (SLAM). The central question SLAM must answer can be stated in a single sentence:

> **The Central Question of This Book**
>
> With no map and no knowledge of its own position, how can a robot build a map, correct its own position, and keep moving, all at the same time?

Unpacked, SLAM is in fact the combination of two subproblems:

- Localization: given a map, estimate the robot's own pose within that map. For example, you open Google Maps on your phone and GPS tells you "you are at the campus gate"—that is localization.
- Mapping: given the robot's pose at every instant, stitch the environmental information measured by the sensors into a map. For example, a surveyor standing at a known station measures the surrounding terrain and draws a topographic map.

Taken separately, both problems already have mature solutions. The difficulty is that a robot in the real world often has neither: on entering a new environment it has no ready-made map to localize against, and no accurate pose with which to map. The spirit of SLAM is to let the robot estimate both unknowns at once—moving, observing, and using "where am I" and "what does the world look like" to correct each other, so that both estimates improve as it goes.

MIT's visual navigation course (VNAV, 16.485) defines SLAM as the problem of estimating, without prior knowledge of the environment and using sensor data alone, both the robot's pose (or its entire trajectory) and a representation of the environment. CMU's 16-833 goes a step further and stresses that SLAM must be carried out in real time under sensing uncertainty—sensors are noisy, wheels slip, lighting changes, and the robot never obtains perfect measurements. SLAM is therefore essentially a problem of making optimal estimates under uncertainty. This is why Part III of this book devotes three full chapters to the mathematical foundations of probability, filtering, and optimization.

Where the Name SLAM Comes From

The term SLAM originates in robotics research on the "stochastic mapping" problem carried out from the late 1980s through the 1990s. Around 1986, Smith, Self, and Cheeseman proposed a framework for estimating robot pose and landmark positions jointly by probabilistic means; subsequently Durrant-Whyte, Leonard, Dissanayake, and others established a complete theoretical foundation using the extended Kalman filter (EKF) and proved that the SLAM problem is convergent—as long as observation continues, the uncertainty in both the map and the localization steadily decreases. After the 2000s, particle filtering (such as Gmapping in Chapter 14

of this book) and graph optimization (such as Cartographer in Chapter 15 and ORB-SLAM in Chapter 22) successively became mainstream, and SLAM moved out of the laboratory into robotic vacuum cleaners, self-driving cars, and smartphone AR, becoming an indispensable core technology of modern intelligent robots.

1.2 Why Localization and Mapping Form a Chicken-and-Egg Problem

To understand why SLAM is hard, the key is to see clearly the circular dependency between localization and mapping:

- To build a map, you must first know where you are. A sensor measures "there is a wall three meters ahead"; this information is meaningful only if the robot's current pose is known—the same measurement will place that wall in completely different parts of the map depending on where the robot stands and which way it faces.
- To localize, you must first have a map. Localization is fundamentally a matter of matching: only by comparing what is seen now against features in the map can the robot infer its own position. Without a map there is nothing to match against.

This is a textbook chicken-and-egg problem: mapping requires localization, localization requires a map, each is a precondition for the other, and neither appears able to go first.

What Happens Without SLAM: Error Accumulation

Some students will ask: doesn't the robot have wheel odometry? If we count how many turns the wheels have made and how far it has traveled, can't we simply add it up

and know the position? In theory yes, but in practice odometry has a fatal weakness: drift. Wheels slip, carpet and tile have different friction, wheel diameters carry manufacturing error, and the small error of every step is accumulated into the position estimate and never disappears on its own. Over ten meters the discrepancy may be only a few centimeters, but after a full loop around one floor of a building the estimated position may already be off by several meters—and if such a pose is fed directly into mapping, the corridors come out curved, the rooms misaligned, and the same wall may even appear twice on the map.

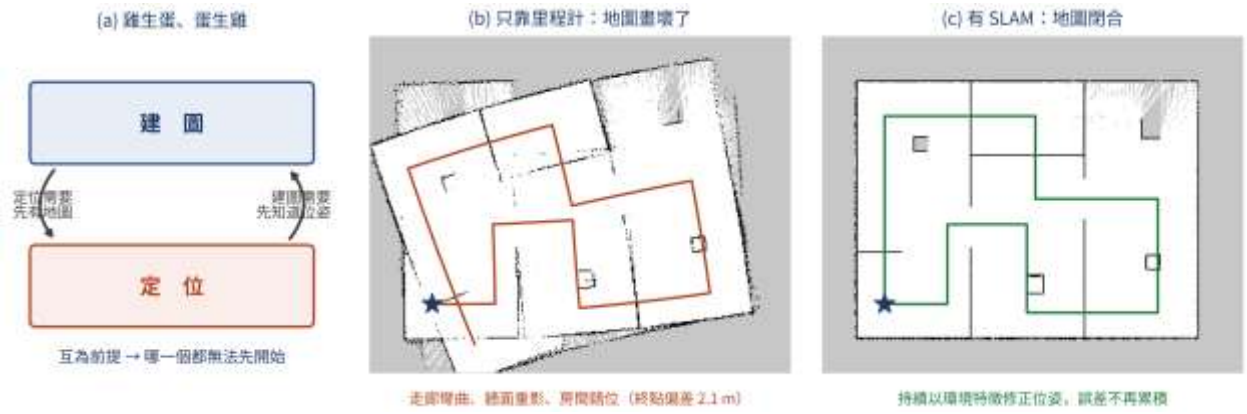
The strategy by which SLAM breaks this deadlock can be summed up in one sentence: do not treat localization and mapping as two sequential steps, but as a single estimation problem in which the two correct each other. Concretely, a SLAM system usually works as follows:

1. Initialization: the robot takes its starting point as the origin. At this moment the map is empty, but the pose is certain (I am at the origin).
2. Motion and prediction: the robot advances a small step and predicts its new pose from wheel odometry or the IMU—this prediction carries error.
3. Observation and update: the sensors measure the surrounding environment; if the measured features agree with features already in the map, the degree of agreement is used in reverse to correct the pose, while newly observed regions are added to the map.
4. Loop-closure correction: when the robot completes a circuit and returns to a place it has already visited, the system recognizes "I have been here before" (loop closure detection). At that moment the accumulated error over the whole

trajectory and the whole map can be distributed and corrected in one pass, so that the map closes up.

In other words, SLAM does not make error vanish by magic; it continually uses the environment itself as a reference standard. Walls do not move and columns do not shift, so as long as the robot repeatedly observes the same environmental features, the drift can be pulled back. This idea runs through the entire book—from Bayesian filtering in Chapter 11 and pose-graph optimization in Chapter 12 to loop closure detection in Cartographer in Chapter 15, the same principle unfolds at different levels.

Figure 1-1 The chicken-and-egg problem, and what a map looks like without SLAM



The left panel draws the circular dependency as a loop: to map you must first know where you are, and to localize you must first have a map to match against. Each is a precondition for the other, and neither appears able to go first.

The middle and right panels show maps of the same robot, the same site, and the same route, built in two different ways. These are genuine simulation runs produced by code: the robot travels one loop along the corridor while a 2D LiDAR scans its surroundings, and the measured wall surfaces are marked into grid cells.

The middle panel uses wheel odometry alone to propagate the pose. The angular error

per step is extremely small (about one hundredth of a degree), and after one loop the endpoint deviates from the start by only 2.1 meters—which does not sound like much, yet the resulting map is already unusable: the whole figure is twisted, the corridor is curved, the same wall is drawn as two lines because the pose differed on the two passes, and the partition walls of the rooms are offset from one another. This is exactly the phenomenon described in the preceding paragraph.

In the right panel the poses have been corrected: the walls are straight, the rooms are square, and the map closes. Note that the sensor data in the two maps are identical—the only difference is where the robot believed itself to be. This sentence is worth remembering: when SLAM fails to produce a good map, the problem is almost always in the localization, not in the sensor.

Three Things to Take from This Figure

1. Map quality depends on pose quality. However accurate the sensor, if the pose is wrong the map is skewed.

2. Error does not appear suddenly; it accumulates step by step—every short segment in the middle panel looks reasonable, and what goes wrong is the sum.

3. SLAM's corrective power comes from the environment itself: walls do not move, so observing the same wall repeatedly pulls the drift back. Chapter 12 will show you how that correction is carried out mathematically.

Two Lines of Solution: A Preview of the Book's Main Thread

Faced with this mutual dependency, researchers over the past forty years have developed two principal lines of solution. They form the main thread from Part III

onward. You do not need to understand the details now, but knowing that these two paths exist will make the later chapters much clearer.

- The probabilistic filtering line: since we cannot be certain where the robot is, we retain many possibilities at once and use sensor data to eliminate implausible hypotheses one by one. This path leads to the Kalman filter and the particle filter, the subject of Chapter 11 and the theoretical basis of Gmapping in Chapter 14.
- The optimization line: write every position the robot has occupied and every constraint measured by the sensors into one enormous system of equations, and solve in a single pass for the configuration that best fits all observations. This path leads to graph optimization and bundle adjustment, the subject of Chapter 12 and the core of Cartographer in Chapter 15 and ORB-SLAM in Chapter 22.

These two lines do not replace each other. Chapter 11 explains the real-time advantage of filtering methods, and Chapter 12 explains why optimization methods can go back and correct historical errors. Chapters 14 and 15 will let you compare the two firsthand at the same site—and when you return to this section afterward, you will have a far more three-dimensional understanding of how the chicken-and-egg problem is solved.

1.3 How a Robot Senses the World: Wheel Odometry, IMU, Camera, LiDAR, Radio, and Geomagnetic Field

Human beings sense "where I am and what is around me" through the eyes, the ears, the vestibular system (the balance organ of the inner ear), and muscular proprioception. A robot needs corresponding senses. The sensors used in SLAM fall broadly into two classes:

- Proprioceptive sensors measure the robot's own state of motion—for example how far the wheels have turned (wheel odometry), or the linear acceleration and angular rate of the body (IMU). They need not observe the outside world, but their error accumulates.
- Exteroceptive sensors measure the environment around the robot—for example the image captured by a camera, the range measured by a LiDAR, the strength of a received radio signal, or the direction of the Earth's magnetic field. They provide an absolute or semi-absolute reference, but are easily disturbed by the environment.

The art of SLAM lies in fusing proprioceptive sensors, which drift but are continuous and smooth, with exteroceptive sensors, which do not drift but are noisy and can fail instantaneously, so that each compensates for the other's weakness. The six major sensor classes examined in depth in later chapters are introduced below.

1. Wheel Odometry

The most basic sensor on a wheeled robot. An encoder mounted on the motor shaft or wheel axle counts wheel revolutions; combined with the wheel diameter and the wheelbase, this yields the distance traveled and the angle turned. Wheel odometry has a high sampling rate, low cost, and continuous data; its drawback is the drift already mentioned—slip, wheel-diameter error, and uneven floors all cause error to accumulate without bound. In ROS, wheel odometry corresponds to the odom coordinate frame and is the source of the motion prediction in nearly every SLAM algorithm (see Chapters 10 and 29).

2. Inertial Measurement Unit (IMU)

An IMU (Inertial Measurement Unit) typically contains a three-axis accelerometer

and a three-axis gyroscope, and some units add a three-axis magnetometer. The accelerometer measures linear acceleration and the gyroscope measures angular rate; integrating them over time can in principle yield velocity, position, and attitude—this is inertial navigation. The IMU's sampling rate is very high (commonly 100 Hz to 1000 Hz) and it is highly sensitive to rapid motion, making it indispensable for unmanned aerial vehicles and for visual-inertial SLAM (such as ORB-SLAM3 in Chapter 23). Integration, however, amplifies noise and bias and produces rapid drift, so an IMU must almost always be fused with other sensors (see Chapter 7).

3. Camera

The camera carries the richest information at the lowest cost: a single image contains texture, color, shape, and even semantics (this is a door, that is a person). A monocular camera is inexpensive but cannot measure depth directly and suffers from scale ambiguity; a stereo camera computes depth from the disparity between the left and right images; an RGB-D camera (such as the RealSense) outputs the range of every pixel directly. SLAM built primarily on cameras is called Visual SLAM and is the main thread of Part VI. The camera's weakness is its dependence on illumination and texture—darkness, backlighting, blank white walls, and motion blur from rapid movement all cause visual SLAM to fail (see Chapters 9 and 20).

4. LiDAR

LiDAR (Light Detection and Ranging) emits laser pulses and measures the time of flight of the returned light, directly yielding precise geometric information of the form "there is something in this direction at this range." A 2D LiDAR scans a single plane by rotating and outputs a ring of range values (a LaserScan); it is standard equipment on robotic vacuum cleaners and teaching platforms. A 3D LiDAR (multi-beam) scans many

elevation angles at once and outputs a three-dimensional point cloud; it is the core sensor of self-driving cars and autonomous inspection robots. LiDAR's strengths are measurement accuracy and immunity to illumination; its drawbacks are higher cost, failure on glass and mirrored surfaces, and noise in rain or fog (see Chapter 8 and Part V).

5. Radio

GPS / GNSS satellite positioning is the best known form of radio positioning, but the signal is blocked indoors, in tunnels, and in urban canyons. Indoors, positioning can use Wi-Fi signal-strength fingerprinting, Bluetooth beacons, or ultra-wideband (UWB). The advantage of radio positioning is that it supplies an absolute position reference and does not drift; its drawbacks are limited accuracy (GPS on the order of several meters, Wi-Fi several meters, UWB ten to thirty centimeters) and susceptibility to multipath reflection. In practice it commonly serves as auxiliary information for SLAM or as a source of initialization (see Chapters 4 and 5).

6. Geomagnetic Field

The Earth is itself a large magnet, and a magnetometer (electronic compass) can measure the direction of the geomagnetic field, providing an absolute heading reference. More interesting still, the rebar and equipment inside a building distort the local magnetic field in characteristic ways, and these distortions form a "geomagnetic fingerprint" that can be used for indoor positioning. Geomagnetic sensing is entirely passive and requires extremely little power; its drawback is susceptibility to interference from ferromagnetic objects, which makes calibration mandatory (see Chapter 6).

Sensor	What It Measures	Principal Strength	Principal Weakness	Chapters
Wheel odometry	Wheel rotation → relative displacement	Inexpensive, continuous, high rate	Slip and drift; error accumulates	Chapters 10, 29
IMU	Acceleration, angular rate	High rate; sensitive to rapid motion	Integration drifts quickly	Chapters 7, 23
Camera	Images (texture, color, semantics)	Information-rich, low cost	Depends on illumination and texture	Chapters 9, 19–23
LiDAR	Laser ranging (2D / 3D)	Geometrically accurate; immune to illumination	Higher cost; fails on mirrored surfaces	Chapters 8, 13–18
Radio	GPS / Wi-Fi / UWB signals	Absolute position; no drift	Limited accuracy; multipath interference	Chapters 4, 5
Geomagnetic field	Direction and magnitude of the Earth's field	Passive; provides absolute heading	Ferromagnetic interference; requires calibration	Chapter 6

Table 1-1 Comparison of Sensors Commonly Used in SLAM

How the Sensors Are Developed in This Book

Each of the six sensor classes mentioned in this section has a dedicated chapter in Part II. The table below is given here so that you know where the in-depth treatment of each sensor can be found, and for convenient reference later.

Table 1-2 Sensors and Their Dedicated Chapters

Wheel odometry → Chapter 3 (dead reckoning), Chapter 13 (fusion with LiDAR), Chapter 29 (implementation and calibration)

IMU → Chapter 7 (principles, drift, attitude filtering, preintegration)

Camera → Chapter 9 (imaging model), Part VI (visual SLAM throughout)

LiDAR → Chapter 8 (principles and specifications), Part IV (2D), Part V (3D)

Radio → Chapter 4 (indoor positioning), Chapter 5 (satellite navigation)

Geomagnetic field → Chapter 6 (electronic compass and geomagnetic fingerprints)

Every chapter has the same structure: principle → specifications and limits → failure scenarios → complementarity with other sensors → hands-on activity.

This consistent structure exists so that you can compare sensors quickly rather than treating them as isolated pieces of knowledge.

One idea that runs through the entire book is planted here: no single sensor is universal. LiDAR is precise but cannot interpret glass; the camera is information-rich but fears darkness; the IMU responds quickly but drifts; satellite signals are accurate but do not reach indoors. Every chapter of Part II discusses honestly the conditions under which that sensor fails, and the experiment in Chapter 30 lets you see with your own eyes how LiDAR and camera each break down at the same site. Understanding

limitations is the core conviction of this book from its first page to its last.

1.4 How SLAM Relates to Autonomous Driving, UAVs, Robotic Vacuums, and AR / VR

Over the past two decades SLAM has gone from an obscure academic topic to a fiercely contested core technology, because it happens to be the common foundation of four major applications.

Autonomous Driving

A self-driving car requires centimeter-level localization accuracy—GPS error of several meters is enough to put the vehicle in the wrong lane. Self-driving systems therefore generally adopt a strategy of "high-definition map plus real-time localization": survey vehicles equipped with 3D LiDAR and cameras first build a high-definition map using SLAM techniques; while driving, the vehicle matches live sensor data against that map to localize precisely. In areas where no high-definition map exists (newly opened roads, parking structures), the vehicle must run SLAM in real time. The 3D LiDAR mapping and the LOAM family of algorithms introduced in Chapters 17 and 18 are precisely the technical backbone of localization and mapping for autonomous vehicles.

Unmanned Aerial Vehicles (Drones / UAVs)

When a UAV flies indoors, beneath bridges, or in tunnels where the GPS signal is blocked (GPS-denied environments), it can localize only with onboard sensors. Because payload and power are severely limited, UAVs favor the lightweight combination of camera plus IMU—that is, visual-inertial odometry (VIO) and visual-inertial SLAM. This

is the specialty of the research groups at ETH Zürich and UZH, and the reason MIT's VNAV course uses UAVs as its experimental platform. After 2015, consumer manufacturers such as DJI built VIO into their products, enabling drones to hover stably indoors; on the academic side the push continues toward smaller and more power-efficient systems, such as NanoSLAM in Chapter 25, which allows a 44-gram micro-UAV to run SLAM entirely onboard.

Robotic Vacuum Cleaners

The robotic vacuum is SLAM's most successful consumer application. Early machines used a random-bounce strategy—turn when you hit a wall—which produced a chaotic cleaning path with both repetition and omission. In 2010 Neato was the first to mount a 2D LiDAR on a vacuum, and in 2017 Xiaomi's first-generation LiDAR vacuum made the technology affordable, bringing SLAM into ordinary households on a large scale for the first time: the robot first builds a map of the room, then plans a boustrophedon path and covers the area zone by zone, greatly improving both efficiency and coverage. The room map, no-go zones, and point-and-clean features you see today when you open a vacuum's app are all SLAM underneath (see Chapter 16).

AR / VR (Augmented and Virtual Reality)

For AR glasses or phone-based AR to pin a virtual object onto a real tabletop, the device must know its own pose in space in real time—this is SLAM, although industry usually calls it "spatial localization" or "inside-out tracking." Apple's ARKit, Google's ARCore, and the headset tracking of Meta Quest are all built on visual-inertial SLAM at their core. VR headsets track environmental features with onboard cameras, replacing the external base stations required by earlier systems. It is fair to say that SLAM is what

turned AR / VR from a seated experience into one in which the user can walk freely.

Conceptual Summary

These four applications look worlds apart, yet underneath they are the same problem: estimating one's own pose in real time and understanding the surrounding space in an unknown or partially unknown environment.

The differences lie only in the sensor suite (LiDAR-centric or camera-centric), the computational resources (a vehicle computer or a phone chip), and the accuracy required (centimeters or decimeters).

This is why the book stresses one set of concepts across many platforms: master the core principles of SLAM and you can transfer freely among self-driving cars, UAVs, robotic vacuums, and AR.

1.5 From the Classroom to the Campus: Teaching Cases for SLAM at the High School and University Level

One distinctive feature of this book is that it grounds the study of SLAM in two real teaching settings: the high school AI and robotics course, and the university intelligent robotics program. The same core concepts are designed into activities of different depth according to the stage of study.

The High School Setting: Understandable and Buildable

At the high school level the emphasis is on building intuition and hands-on experience rather than rushing into derivations. Taking the classroom as the experimental site, the following progressive activities can be designed:

- Activity 1: Human SLAM. One student is blindfolded and directed verbally by classmates to walk around the classroom. While walking, that student narrates "where I think I am," and the others record on paper the gap between the actual and estimated positions, experiencing how error accumulates in the absence of external observation.
- Activity 2: First encounter with phone sensors. Use compass and accelerometer apps on a phone to observe sensor readings and understand the principles and the noise of inertial and geomagnetic sensing.
- Activity 3: Observing a robotic vacuum. Observe the cleaning path of a household robotic vacuum and the map in its app, and discuss how it knows which areas it has already cleaned.
- Activity 4: Mapping in simulation. Drive a virtual robot in a ROS simulation environment (such as the TurtleBot3 simulator) to build a map of the classroom, experiencing the complete SLAM workflow without any physical hardware.

The University Setting: Able to Derive, to Implement, and to Research

At the undergraduate and capstone level, the university intelligent robotics laboratory and the campus become the site, and the same set of concepts is driven deeper and extended further:

- Foundational implementation: run Gmapping and Cartographer with a 2D LiDAR teaching platform in the classroom and corridor (Chapters 14, 15, and 29) and compare the map quality of the two algorithms quantitatively.

- Advanced implementation: run RTAB-Map and ORB-SLAM3 with an RGB-D camera (Chapter 30) and compare the performance of visual SLAM and LiDAR SLAM at the same site.
- Capstone research: quadruped campus inspection, tightly coupled LiDAR-inertial mapping, vision-guided UAVs, and multi-robot collaborative mapping (Chapters 18, 22, and 24–26), connecting to graduate-level topics.

Chapter 31 of this book provides six complete capstone projects together with a grading rubric, covering the full spectrum from high school to university capstone work. Instructors are encouraged to show students this "learning map" at the beginning of the term, so that they know which visible, tangible robotic system each concept they are learning will ultimately lead to.

1.6 SLAM System Outputs: Trajectory, 2D Map, 3D Point Cloud, Semantic Map, and Topological Map

What exactly does a SLAM system produce when it has finished running? Depending on the needs of the application, the output takes one of five forms. Understanding these forms will help you quickly grasp, in later chapters, which kind of output a given algorithm is designed to produce.

1. Trajectory

The sequence of poses at every instant from start to finish. The trajectory is the most direct basis for evaluating SLAM accuracy: comparing the estimated trajectory against the true trajectory (ground truth, usually supplied by a motion capture system or high-precision GPS) yields the absolute trajectory error (ATE) and the relative pose error (RPE). Public datasets such as EuRoC, TUM, and KITTI (Chapters 26 and 30) all

provide ground-truth trajectories for benchmarking.

2. 2D Occupancy Grid Map

The floor is divided into cells of fixed size (for example 5 cm per cell), and each cell records the probability that it is occupied: black denotes obstacles (walls, table legs), white denotes free, traversable space, and gray denotes unknown regions not yet observed. This is the map form most commonly used by indoor wheeled robots and robotic vacuums, and it is the standard input to the ROS navigation stack (Chapter 13).

3. 3D Point Cloud

A spatial model composed of hundreds of thousands to hundreds of millions of three-dimensional points, each carrying coordinates (x, y, z) and optionally color or reflectance intensity. A 3D point cloud represents volumetric structure completely—staircases, ceiling conduits, tabletop heights—and is the basis for autonomous inspection, factory digital twins, and high-definition maps for self-driving cars (Chapter 17).

4. Semantic Map

On top of the geometric map, a "meaning label" is attached to each region or object: this patch is a door, that one a table, this is a corridor, that is a charging station. A semantic map lets the robot know not merely where the obstacles are but what those obstacles are, so that it can carry out tasks stated in human language such as "go to the registrar's office" or "avoid the stairs." The semantic map is where AI vision models (YOLO, SAM, VLM) meet SLAM, and it is the subject of Part IX (Chapters 32–34).

5. Topological Map

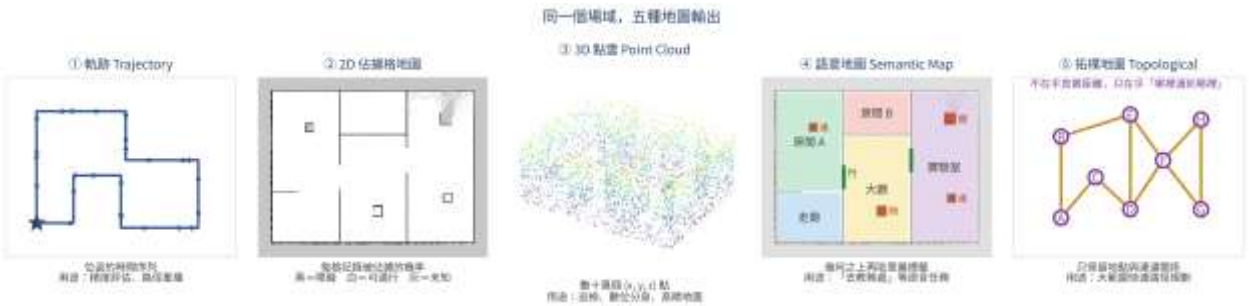
Discarding precise geometric coordinates, this form retains only places and the

connectivity between them, much as a metro route diagram ignores the true distances between stations and cares only about which station connects to which. A topological map requires very little memory and supports extremely fast path planning, which suits navigation over large areas; practical systems often adopt a hybrid map in which the topology is the skeleton and the grid map is the flesh.

Output Form	Data Structure	Typical Application	Representative Chapters
Trajectory	Time series of poses	Accuracy evaluation, path replay	Chapter 31
2D occupancy grid map	Probabilistic grid	Indoor navigation, robotic vacuums	Chapters 13–16
3D point cloud	Set of three-dimensional points	Spatial scanning, autonomous inspection, self-driving cars	Chapters 17, 18
Semantic map	Geometry + object labels	Semantic navigation, task-oriented robots	Chapters 24, 32
Topological map	Graph of nodes and edges	Fast planning over large areas	Chapters 3, 32

Table 1-3 The Five Output Forms of a SLAM System

Figure 1-2 Five Map Outputs for the Same Site



These five panels come from the same simulated site and the same scanning run; they differ only in what is kept and what is discarded. (1) The trajectory retains only the

robot's own sequence of poses and records nothing of the environment. (2) The occupancy grid map divides the floor into cells and records the occupancy probability of each cell; it is the protagonist of Part IV. (3) The point cloud retains three-dimensional structure and reveals wall heights and ceilings; it is the foundation of Part V. (4) The semantic map overlays meaning on geometry—this is a corridor, that is a laboratory, that block is a table; it is the subject of Chapter 32. (5) The topological map discards all geometry and keeps only places and their connectivity, like a metro route diagram.

The trade-off in information content deserves attention. Going from (1) to (5) is not a progression from worse to better—a topological map may hold one millionth of the data of a point cloud, but it is of no help whatsoever in steering a robot around a chair; conversely, for planning a route from the first-floor lobby to a third-floor laboratory, a point cloud is far too cumbersome. Practical systems commonly maintain several map types at once, using a topological map for global planning and a grid map for local obstacle avoidance—this is the "topology as skeleton, grid map as flesh" mentioned in Section 1.6.

You may treat this figure as a checklist of goals for the book: after Part IV you will have built (2) with your own hands, Part V gives (3), the visual SLAM of Part VI produces feature maps and variants of (3), and Chapter 32 produces (4). It does not matter if the details are unclear now; simply form the impression that SLAM has more than one kind of output.

Formulations and Procedures

1. The Probabilistic Definition of SLAM

Although the rigorous derivation waits until Chapter 11, this chapter states the

standard mathematical formulation of SLAM so that you have a precise anchor for what SLAM is estimating. Define the following symbols:

- x(t): the robot pose at time t, which in the 2D case is (x, y, θ).
- m: the map of the environment (which may be a grid map, a set of landmarks, or a point cloud).
- z(1:t): all sensor observations from time 1 to t (laser scans, image features, and so on).
- u(1:t): all control inputs or odometry readings from time 1 to t.

The full SLAM problem is to estimate the joint probability distribution over the entire trajectory and the map, given all observations and controls:

$$p(x(1:t), m \mid z(1:t), u(1:t))$$

The online SLAM problem estimates only the current pose and the map, marginalizing out the past trajectory, which suits real-time systems:

$$p(x(t), m \mid z(1:t), u(1:t))$$

In plain language: the answer SLAM seeks is "given everything I have seen and every path I have traveled, what are the most likely trajectory and map?" The reason it is written as a probability distribution rather than a single value is precisely that sensors are noisy and motion is imperfect—the answer SLAM returns always carries information about how certain it is. Particle-filter SLAM (Chapter 14), EKF-SLAM (Chapter 11), and graph-optimization SLAM (Chapter 12) are all, in essence, different numerical methods for solving this probabilistic problem.

2. The Standard Pipeline of a SLAM System

Modern SLAM systems generally adopt a front-end plus back-end architecture; the

pipeline is shown in Figure 1-3:

Figure 1-3 The Standard Pipeline of a SLAM System

Sensor data (laser scans / images / IMU / wheel odometry)

↓

Front end: feature extraction, data association, scan matching, odometry estimation

↓

Back end: filtering or graph optimization to estimate the optimal poses and map

↓

Loop closure: recognize revisited places and correct globally accumulated error

↓

Output: trajectory + map (2D grid map / 3D point cloud / semantic map)

The front end is responsible for turning raw sensor data into geometric constraints, and the back end for finding the most consistent answer under all those constraints, while loop closure detection is the key weapon against long-term drift. Part II of this book covers the sensors (the input to the front end), Part III the mathematics (the tools of the back end), and Parts IV through VI how various concrete systems realize this pipeline; from Part VII onward the book examines frontier extensions into miniaturization, multi-robot collaboration, and AI-based semantics.

1.7 The Technical Evolution of SLAM and the Contemporary Frontier

The preceding sections introduced the basic problem and the constituent parts of SLAM. This section widens the view to how the field has evolved over forty years and what is happening right now—which will help you understand where on that line of development each technique taught in the later chapters sits.

Evolution in Four Stages

Synthesizing recent survey literature, the development of SLAM can be divided roughly into four stages, and each stage has not replaced the one before it but has been layered onto it and reinforced it:

1. The hand-designed era (1986 to the 2010s): researchers used human understanding of geometry to hand-design feature detectors (such as SIFT and ORB in Chapter 20) and filtering algorithms (Chapter 11). This era established the mathematical skeleton of SLAM and is the principal content of Parts III through VI of this book.
2. The deep learning era (from about 2015): neural networks replaced some hand-designed modules—learned features, learned depth estimation, learned loop closure detection. The monocular depth estimation of Section 9.4 and the learned features of Section 20.7 both belong to this stage.
3. The radiance field era (from 2021): neural radiance fields (NeRF) and 3D Gaussian splatting (3DGS) redefined what a map is—a map is no longer merely a point cloud or a grid, but a field from which photorealistic images can be re-rendered.
4. The foundation model era (from 2023): vision-language models and robot

foundation models entered SLAM, moving robots from knowing shape to understanding meaning. This is the subject of Chapter 32.

How Radiance Fields Changed the Definition of a Map

The transition at the third stage deserves particular comment, because it changed a premise that had seemed self-evident: what a map should look like.

The map in traditional SLAM is an occupancy grid (Chapter 13) or a point cloud (Chapter 17)—these answer where something is, but cannot answer what the scene should look like when viewed from a given angle. NeRF represents the radiance field of the whole space implicitly with a neural network and can render photorealistic images from arbitrary viewpoints; 3DGS instead adopts an explicit representation, modeling the scene as a collection of three-dimensional Gaussian primitives, each with a position (mean), a covariance, a color, and an opacity.

The reason 3DGS rose so quickly in robotics is efficiency. Unlike implicit representations, which require querying a neural network at every point in space, the differentiable rasterization pipeline of 3DGS overcomes the bottleneck of slow training and rendering in large neural networks—precisely the obstacle that made NeRF-based systems difficult to use on robots. In addition, its explicit structure allows the map to be edited or deformed easily, which is a considerable advantage for map correction and adaptation to changing environments.

Clarification: Trade-offs Among Three Scene Representations

[Implicit] Approximates the radiance field with a neural network. Memory is small, but every query must run the network, so it is slow.

[Explicit] Renders directly from learned spatial features (voxels, hash grids, Gaussian primitives) with no neural component. Fast, but memory-hungry.

[Hybrid] Combines learned spatial features with a neural network. Both training and rendering are accelerated, and memory demand is likewise high.

For a robot the trade-off between speed and memory is always present—this is the same problem as the trade-offs of the milliwatt-class platforms in Chapter 25,

transposed to the level of map representation.

Four Directions at the Contemporary Frontier

Surveying the research of the past two years, the most active directions in SLAM can be grouped into four, and they make a practical map for any student who intends to go into research later:

- Robustness: enabling radiance-field SLAM to cope with dynamic environments and motion blur. Recent work includes dense neural SLAM designed specifically for blurred input and Gaussian mapping methods for evolving scenes; some systems also use the IMU to handle blur, rolling shutter, and severely noisy thermal imagery—which echoes the triple blow of high-speed flight discussed in Section 26.4.
- Collaboration: multiple robots building a Gaussian map together. It is worth noting that conventional collaborative SLAM has trended toward decentralization for scalability and robustness, yet because training three-dimensional Gaussians is computationally expensive, collaborative Gaussian

SLAM today is predominantly centralized—an instructive contrast with the architectural discussion in Section 27.3, and an illustration of how technology choices are governed by computational cost.

- Language: incorporating language capability into the map representation so that a robot can be queried about space in natural language. This is the subject of Chapter 32 and an extension of VLM navigation in Section 24.5.
- Foundation models: supporting monocular Gaussian SLAM with multi-view foundation models, or addressing fundamental questions such as what the best three-dimensional scene representation for a robot actually is. This line is pushing geometric SLAM and artificial intelligence toward the same meeting point.

Where This Book Stands

Having read this far, you may ask: if radiance fields and foundation models are so prominent, why does the greater part of this book still treat classical geometric methods?

The answer has three layers. First, practicality: the overwhelming majority of systems actually deployed in products today—robotic vacuums, self-driving cars, inspection quadrupeds, AR glasses—still have the geometric methods taught in this book as their backbone. They are mature, interpretable, and have well-defined computational requirements.

Second, foundation: radiance fields and foundation models have not replaced geometry; they are built on top of it. 3DGS SLAM still requires pose estimation, still requires loop closure correction, and still must contend with sensor limitations. Without understanding geometry, one cannot judge when these new methods will fail.

Third, pedagogy: the aim of this book is to train engineers who can exercise judgment, not users who chase terminology. The capstone projects of Chapter 31 and the discussion of Chapter 32 repeatedly stress the same point—technology will keep evolving, but the ability to understand limitations and choose the appropriate tool does not go out of date.

Case Studies

Case 1-1 The Xiaomi Robotic Vacuum: SLAM Enters Ordinary Homes

In 2017 Xiaomi released the first-generation Mijia robotic vacuum, which carried a rotating 2D LiDAR on top yet sold for less than half the price of conventional high-end machines. Its workflow is a complete demonstration of the SLAM loop introduced in this chapter: on start-up the LiDAR scans several thousand range points per second; as the robot drives it builds an occupancy grid map of the room with a particle filter algorithm while simultaneously localizing itself within that map; once the map is complete, the path planning module divides the rooms into zones and covers each zone with a boustrophedon path, ensuring neither repetition nor omission. This machine brought the term SLAM, which until then appeared only in research papers, into ordinary living rooms on a large scale for the first time. The lesson for the learner is this: SLAM is not some unreachable frontier technology—its core version runs on a processor of smartphone class, which is also why this book believes high school students can learn SLAM.

Case 1-2 The DARPA Subterranean Challenge: SLAM's Ultimate Proving Ground

From 2018 to 2021 the U.S. Defense Advanced Research Projects Agency (DARPA) held the Subterranean Challenge, which required robot teams to explore autonomously, build maps, and search for target objects in man-made tunnels, underground urban spaces, and natural caves. The underground environment combines every nightmare of SLAM: no GPS at all, dust and smoke that corrupt LiDAR returns, darkness that disables cameras, and narrow passages and vertical shafts that tax mobility. The competing teams—including groups from MIT, CMU, and ETH—generally used multi-sensor fusion of LiDAR, IMU, camera, and thermal imager, and formed heterogeneous teams of wheeled robots, quadrupeds, and UAVs that mapped collaboratively. The competition greatly advanced the practical maturity of LiDAR-inertial SLAM (Chapter 18) and multi-robot SLAM (Chapter 27), and it proved one thing: SLAM with a single sensor on a single robot is approaching maturity, and robustness in unknown, severe environments is the main battleground of the coming decade.

Case 1-3 The AR Measuring App on a Phone: Visual-Inertial SLAM in Your Pocket

Open the Measure app built into the iPhone, point the camera at a tabletop, move for a few seconds, and you can draw a virtual ruler in midair that conforms to the real world. Behind this is ARKit's visual-inertial SLAM: the phone camera tracks image feature points at 30 to 60 frames per second, the IMU measures motion several hundred times per second, and fusing the two yields a real-time estimate of the phone's six-degree-of-freedom pose together with a reconstruction of the planar structure of the tabletop. This case makes an excellent first classroom demonstration—no robot hardware is needed, since an industrial-grade SLAM system already lives inside every student's phone. Instructors can have students experiment with how the

ruler loses accuracy in front of a blank white wall (no texture), in a dim room, or when the phone is swung quickly, experiencing firsthand the camera limitations described in Section 1.3.

Case 1-4 From iMAP to 3DGS: What the Map Became in Five Years

The case studies in this book usually treat applications of mature technology; this one is an exception—it treats something now unfolding, so that you can feel the pace of change in this field.

In 2021 a work called iMAP demonstrated for the first time that real-time SLAM with a neural implicit representation was possible, and it is regarded as the starting point of radiance-field SLAM. In the few years since, related research has grown explosively: some work improved it with voxel feature grids, some switched to feature planes, and some combined point-cloud-based radiance fields. By the end of 2023, 3D Gaussian splatting had appeared and, thanks to its explicit representation and real-time rendering, rapidly became the new mainstream direction.

The speed of this change deserves attention: from a proof of concept to the research mainstay of an entire community took roughly three years. By contrast, ORB-SLAM, the protagonist of Chapter 22, was published in 2015 and remains the standard benchmark for visual SLAM to this day—undiminished after a decade.

The contrast raises a question worth pondering: what kind of technology endures, and what kind is quickly superseded? The author's observation is that methods which solve fundamental problems (epipolar geometry, graph optimization, loop closure detection) have very long lifetimes, whereas methods which solve questions of representation (in what format the map is stored, how the image is rendered) iterate

rapidly. This is also why the greater part of this book is devoted to the former.

The advice to students is this: understand each new technique as yet another representation, and ask what limitation of the older methods it removes and what price it pays (memory? computation? robustness?). With that framework, whatever new term appears next year, you will be able to place its value quickly.

Hands-On Activity

Activity 1-1 The World as the Sensor Sees It vs. the Actual Map

Purpose: to experience the gap between recording an environment from a first-person viewpoint and viewing it as a map from above, and to understand the data transformation problem that SLAM must solve.

Equipment

- One smartphone (for video recording), or one wheeled teaching robot.
- Two sheets of graph paper, a pencil, and a ruler (or tape measure).

Procedure

1. Form groups of three to four and choose an indoor route: for example, start at the rear door of the classroom, go straight along the corridor, turn right, pass the water fountain, and return to the front door.
2. First-person recording: one student holds the phone at walking height and records video over the entire route, narrating the landmarks seen along the way ("there is a fire extinguisher on the left," "a glass door ahead").
3. Draw a map from the record: back in the classroom, with the video as the only permitted reference and no return to the site, the group works together to

draw on graph paper the world as the sensor saw it—the route, the corners, the relative positions of landmarks—annotated with estimated distances.

4. Field measurement: return to the site with the tape measure and measure the corridor length, the positions of the corners, and the spacing between landmarks, then draw the actual map on the second sheet of graph paper.
5. Comparison and discussion: overlay the two drawings, measure where the error is largest, and discuss where the error came from—a misjudged sense of distance? an incorrectly estimated turning angle? landmarks recorded out of order?

Discussion Questions

1. Did the error in your drawn map grow larger or smaller the farther you traveled? How does this relate to the drift described in the text?
2. If the route returns to its starting point, did the endpoint of your drawing coincide with the start? If not, how should the whole map be corrected? Which term in the text does this operation correspond to?
3. Which landmarks in the video were easiest to recognize and which were easiest to confuse? What does this suggest about the kind of environment camera-based SLAM prefers?

Extended Challenge (Undergraduate)

Using the video recorded on the phone as input, run an open-source visual SLAM system on a computer (for example ORB-SLAM3 in monocular mode), output the estimated trajectory, and compare it three ways against your hand-drawn map and the surveyed map. This challenge previews the content of Chapters 22, 23, and 30.

Further Reading

1. C. Cadena et al., "Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age," IEEE Transactions on Robotics, vol. 32, no. 6, 2016. (The authoritative survey of thirty years of SLAM; cited repeatedly throughout this book.)
2. H. Durrant-Whyte and T. Bailey, "Simultaneous Localization and Mapping: Part I & Part II," IEEE Robotics & Automation Magazine, 2006. (The classic introductory tutorial on SLAM; establishes the theoretical foundation.)
3. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005. (The bible of probabilistic robotics and the principal reference for the mathematical foundations in Part III.)
4. MIT OpenCourseWare, 16.485 Visual Navigation for Autonomous Vehicles (VNAV). (A complete, freely available course on visual navigation, with lecture notes and labs.)
5. CMU 16-833 Robot Localization and Mapping course website. (A graduate SLAM course organized around state estimation and optimization.)
6. Cyrill Stachniss (University of Bonn), Robot Mapping lecture series, freely available on YouTube. (Explains SLAM algorithms with intuitive animations; well suited to self-study.)
7. C. Campos et al., "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM," IEEE Transactions on Robotics, 2021. (The original paper on the protagonist of Chapter 23.)

A Guide to the Whole Book: A Map of Thirty-Four Chapters

As a closing to Chapter 1, a bird's-eye view of the whole book is given here. You need

not memorize it now, but whenever you feel lost later, you can come back and see which stage of the journey you are on.

Figure 1-4 The Learning Path Through the Nine Parts of the Book



Three Possible Reading Routes

- High school / introductory route: Chapters 1–3 to establish the concepts → Chapters 8 and 9 to learn about LiDAR and cameras → Chapters 13 and 16 to understand 2D mapping and robotic vacuums → Chapters 28 and 29 for hands-on simulation. No deep mathematics is required anywhere along the way, and most of the hands-on activities in each chapter can be completed in simulation.
- Full undergraduate route: read Parts I through VI in order, carrying out the implementation chapters of Part VIII in parallel. The mathematics of Part III is the critical threshold; pairing it with the hands-on activities of Chapters 11 and 12 is recommended for building intuition.

- Capstone / research route: skim Parts I through III quickly to establish a common vocabulary, then go deep according to your research direction—Part V for LiDAR, Part VI for vision, Part VII for miniature platforms, Part IX for AI integration—and converge with the project planning of Chapter 31.

Whichever route you choose, the rhythm of study this book recommends is the same: build intuition first, then understand the principle, then verify by doing, and finally seek out the failure boundary deliberately. Section 28.7 expands this rhythm into five concrete stages.

Summary

1. The core problem SLAM (simultaneous localization and mapping) must solve is this: with no map and no knowledge of its own position, a robot must move, build a map, and correct its own pose, all at the same time.
2. Localization requires a map and mapping requires localization, so the two form a chicken-and-egg circular dependency. SLAM's solution is to treat them as a single estimation problem in which observations of the environment continually correct both, with loop closure detection used to distribute long-accumulated error.
3. A robot's sensors divide into proprioceptive sensors (wheel odometry and IMU: continuous but drifting) and exteroceptive sensors (camera, LiDAR, radio, geomagnetic field: providing an environmental reference but each with its own weakness). The essence of SLAM is sensor fusion in which each compensates for the other's shortcomings.

4. The four major applications—autonomous driving, UAVs, robotic vacuums, and AR / VR—share the same SLAM foundation, differing only in sensor suite, computational resources, and accuracy requirements.
5. The output of a SLAM system takes five forms—trajectory, 2D occupancy grid map, 3D point cloud, semantic map, and topological map—each corresponding to different application needs.
6. Mathematically, SLAM estimates the joint probability distribution over the trajectory x and the map m given the observations z and the controls u; modern systems solve it with a front-end / back-end / loop-closure architecture.

In the next chapter we visit the SLAM courses of world-renowned universities including MIT, CMU, Stanford, ETH / UZH, and Bonn / Freiburg, to see how leading institutions arrange the learning path for this subject, and to distill from them this book's pedagogical main thread of sensors → mathematics → front end → back end → loop closure → capstone.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 2

A Guided Tour of SLAM Courses at World-Renowned Universities

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 1 established what problem SLAM addresses; this chapter answers a practical question: how do the best universities in the world teach this subject? This is more than a broadening of horizons—the overall architecture of this book is distilled from the common skeleton of those courses, so understanding this chapter is equivalent to holding the key to the organizing logic of the twenty-nine chapters that follow.

Figure 2-0 How Chapter 2 Connects to the Rest of the Book

[Building on the previous chapter] The six sensors, the chicken-and-egg problem, and the five map representations of Chapter 1 all have counterpart units in the courses surveyed here.

[This chapter is responsible for] touring the SLAM courses of five leading universities and distilling their common architecture.

[Where it leads]

2.1 MIT VNAV → IMU preintegration in Chapter 7, manifold optimization in Chapter 12, visual-inertial SLAM in Chapter 23

2.2 CMU 16-833 → probabilistic filtering in Chapter 11, factor graphs and GTSAM in Chapter 12

2.3 Stanford → navigation system architecture in Chapter 3, planning and control in Chapter 29

2.4 ETH / UZH → camera models in Chapter 9, epipolar geometry in Chapter 21, UAV VIO in Chapter 26

2.5 Bonn / Freiburg→ particle filtering in Chapter 11, occupancy grids in Chapter 13, Gmapping in Chapter 14

2.6 The common architecture → the basis on which the nine parts of this book are organized

[Prerequisites] None. This chapter is a survey and involves no mathematical

derivation.

Learning Objectives

A craftsman who wishes to do his work well must first sharpen his tools; a student who wishes to go deep must first survey the road. This chapter lets you stand on the shoulders of giants by touring the SLAM-related courses of five leading universities and seeing how first-rate institutions arrange the learning path for this subject. After completing this chapter, you should be able to:

7. Name the SLAM course, the principal instructor, and the pedagogical focus at each of the five academic centers: MIT, CMU, Stanford, ETH / UZH, and Bonn / Freiburg.
8. Compare the entry points taken by these courses—visual navigation, probabilistic inference, system integration, visual-inertial estimation, and probabilistic mapping—and explain their similarities and differences.
9. Summarize the common architecture of SLAM curricula: sensors → mathematics → front end → back end → loop closure → capstone, and map it onto the nine parts of this book.
10. Know where the free learning resources published by each school (OpenCourseWare, YouTube lecture videos, public lecture notes) can be found and how to use them.
11. Plan a self-study route through SLAM that suits your own level and goals.

Core Concepts

Keyword	English / Designation	Explanation
視覺導航	Visual Navigation	Robot navigation with the camera as primary sensor; the main thread of the MIT VNAV course.
狀態估測	State Estimation	The mathematical methods for inferring system state (pose, velocity, map) from noisy measurements.
機率圖模型	Probabilistic Graphical Model	A model expressing relations among random variables as nodes and edges; the theoretical language of the modern SLAM back end.
因子圖	Factor Graph	A probabilistic graphical model that represents measurements as factors; the core tool of CMU 16-833.
視覺里程計	Visual Odometry, VO	Estimating camera motion frame by frame from images alone; the final project of the UZH course.
視覺慣性里程計	VIO	Odometry fusing camera and IMU; the core technology of phone AR and UAVs.
流形最佳化	Manifold Optimization	The mathematical technique of optimizing on curved spaces such as the rotation group; a signature topic of MIT VNAV.
佔據格建圖	Occupancy Grid Mapping	The classic mapping method systematized by the Freiburg / Bonn courses.
開放式課程	OpenCourseWare, OCW	Complete course materials published free by a university; MIT OCW is the best-known example.
大規模開放線上課程	MOOC	Online courses on platforms such as edX and Coursera; the ETH Autonomous Mobile Robots course has a MOOC

Keyword	English / Designation	Explanation
		version.

2.1 MIT VNAV: Visual Navigation, Self-Driving Cars, UAVs, and Optimization

Course 16.485, Visual Navigation for Autonomous Vehicles (VNAV), offered by the Department of Aeronautics and Astronautics at MIT and taught by the SPARK Lab led by Professor Luca Carlone, is one of the most representative graduate courses in visual navigation anywhere in the world. Professor Carlone is himself a leading figure in SLAM—he is one of the principal authors of the 2016 SLAM survey listed in the Further Reading of Chapter 1.

Course Thread and Distinctive Features

The design of VNAV follows a clear logical line: begin from geometry, converge through optimization, and validate on UAVs. The content falls roughly into four blocks:

- Foundations of three-dimensional geometry: representations of rotation (rotation matrices, quaternions), rigid-body transformations, camera models, and two-view geometry—corresponding to Chapters 10 and 21 of this book.
- Optimization on manifolds: a rotation is not an ordinary vector and cannot simply be added or subtracted; optimization must be carried out on a curved space called a manifold. This is VNAV's most distinctive mathematical topic and the key to understanding modern SLAM back ends—Chapter 12 of this book introduces its spirit in more accessible terms.
- The visual front end: feature extraction and matching, visual odometry, place recognition, and loop closure detection—corresponding to Chapters 20 through 22 of this book.

- SLAM back end and systems: pose-graph optimization, bundle adjustment, visual-inertial fusion, and frontier extensions into semantic understanding—corresponding to Chapters 12, 23, and 32.

Pedagogy: Team Projects with UAVs

The most striking feature of VNAV is the arrangement of its laboratory work: students work in teams on real UAVs, completing week by week a sequence of experiments from attitude control and trajectory tracking through visual features and visual odometry to a complete navigation system, and closing the term with a competition-style project. This design, in which the mathematics of each week is immediately validated on a flying vehicle, is precisely the spirit emulated by the implementation chapters of Part VIII of this book.

The complete VNAV materials (lecture notes, assignments, and lab guides) are freely available on MIT OpenCourseWare (course 16.485, Fall 2020), making it one of the most complete free resources for self-study in visual navigation. Readers of this book are advised to take on the VNAV notes after finishing the mathematical foundations of Part III, which will make the effort go considerably further.

2.2 CMU Robot Localization and Mapping: Localization, Mapping, and Back-End Optimization

The Robotics Institute at Carnegie Mellon University (CMU) is the oldest and largest robotics research institution in the world; its course 16-833, Robot Localization and Mapping, is taught by Professor Michael Kaess. Professor Kaess is a representative figure in SLAM back-end algorithms—the iSAM / iSAM2 incremental smoothing algorithms and the GTSAM library, widely used in modern graph-optimization SLAM,

come from the research group he shares with Professor Frank Dellaert.

Course Thread and Distinctive Features

The course description of 16-833 states at the outset that localization and mapping are foundational capabilities for mobile robots operating in the real world, and that more challenging than either problem alone is their combination—SLAM; a robust solution must handle the uncertainty inherent in sensor measurements while providing localization and map estimates in real time. The course therefore centers on efficient probabilistic inference algorithms at the intersection of linear algebra and probabilistic graphical models, covering:

- The Bayes filter family: the Kalman filter, the extended Kalman filter (EKF-SLAM), the particle filter, and FastSLAM—corresponding to Chapters 11 and 14 of this book.
- Smoothing and graph optimization: factor graphs, least squares, and the incremental iSAM solver—corresponding to Chapter 12.
- Data association and loop closure: which landmark in the map does a given measurement actually correspond to? This is the step at which SLAM most easily goes wrong.
- State-of-the-art systems: in the later part of the course students read contemporary SLAM system papers and see how theory is reduced to practice.

A Comparison with MIT VNAV

If VNAV enters from vision and geometry, 16-833 enters from probability and inference: VNAV devotes substantial time to cameras and manifolds, while 16-833 treats filtering, factor graphs, and sparse linear algebra exhaustively. Taken together, the two courses cover the complete knowledge map of the modern SLAM front end

and back end—which is also why this book separates Part II (sensors) from Part III (mathematics) before integrating them into systems in Parts IV through VI.

2.3 Stanford Principles of Robot Autonomy: Localization, Perception, SLAM, and Planning

Principles of Robot Autonomy (course AA274A / CS237A) at Stanford University is offered jointly by the Department of Aeronautics and Astronautics and the Department of Computer Science and taught by the Autonomous Systems Laboratory (ASL) led by Professor Marco Pavone and colleagues. Unlike the previous two courses, this one is positioned not to dig deep into SLAM as a single topic but to walk through the entire system of an autonomous robot.

Course Thread and Distinctive Features

The Stanford course covers the complete technical stack of an autonomous robot, within which SLAM is the link that connects what comes before to what comes after:

- Motion and control: kinematic models, trajectory tracking, and feedback control—the basis of a robot that can move.
- Perception: camera models, image features, object detection, and machine learning for perception—a robot that can see.
- Localization and SLAM: Bayes filtering, Monte Carlo localization, and EKF-SLAM—a robot that knows where it is.
- Planning and decision making: path planning (A*, RRT), trajectory optimization, and task-level decision making—a robot that knows where to go and how to get there.

- System integration: ROS serves as the backbone all term, and at the end students integrate a complete autonomous navigation system in simulation and on a physical robot.

What This Means for the Present Book

Stanford's design answers an important question: what is SLAM for once you have learned it? The answer is that SLAM is the hub of an autonomous system, taking in sensor data from below and supporting path planning and task execution above. The navigation system overview of Chapter 3 and the navigation stack of Chapter 29 in this book adopt exactly this systems view, so that you always know what role the algorithm in front of you plays in the robot as a whole. This system-first, algorithm-second pedagogy suits project-based courses at the high school and undergraduate level particularly well.

2.4 ETH Zürich / UZH: Visual-Inertial Odometry, VIO, and MAV Navigation

ETH Zürich and the University of Zurich (UZH) are geographically adjacent, share faculty, and together form the heart of mobile robotics research in Europe. Two complementary course lineages are found here, representing the systems school and the vision school respectively.

ETH: Autonomous Mobile Robots

The Autonomous Systems Lab (ASL) at ETH, led by Professor Roland Siegwart, has long offered the course Autonomous Mobile Robots, whose textbook of the same name (MIT Press, by Siegwart, Nourbakhsh, and Scaramuzza) is one of the most widely used texts in mobile robotics, covering kinematics, perception, localization, and

navigation planning. The course is also offered as a MOOC on edX, free to learners worldwide. In addition, ASL's research results in visual-inertial estimation—open-source systems such as ROVIO and OKVIS—have become important infrastructure for localization on UAVs and mobile devices; we will meet them again in Chapters 23 and 26.

UZH: Vision Algorithms for Mobile Robotics

Professor Davide Scaramuzza of the Robotics and Perception Group (RPG) at UZH is an international leader in visual odometry and event camera research. His course Vision Algorithms for Mobile Robotics covers image formation, filtering, feature extraction, multi-view geometry, dense reconstruction, tracking, image retrieval, event-based vision, visual-inertial odometry, and SLAM—and the course website states outright that these are the algorithms behind Apple ARKit, Microsoft HoloLens, Meta Quest, and the NASA Mars helicopter.

The most elegant feature of this course is its building-block final project: each lecture is followed by a matching exercise in which students implement one module of a visual odometry pipeline (feature extraction, matching, pose estimation, triangulation, and so on), and at the end of the term all the blocks are assembled into a complete, runnable visual odometry pipeline validated on real datasets such as KITTI. All lecture videos and notes are freely available on the RPG website and YouTube. The knowledge structure of Chapter 9 and Chapters 20 through 21 of this book draws heavily on the ordering used in this course.

2.5 University of Bonn / Freiburg: Robot Mapping and the SLAM Courses of Cyrill Stachniss

The University of Freiburg and the University of Bonn in Germany are the founding centers of probabilistic robotics. Professor Wolfram Burgard (a co-author of Probabilistic Robotics) long taught Introduction to Mobile Robotics and Robot Mapping at Freiburg; his student Professor Cyrill Stachniss subsequently continued and extended that teaching line at the University of Bonn with a complete course centered on SLAM and environmental modeling.

Course Thread and Distinctive Features

The great strength of the Freiburg / Bonn line is that it presents probabilistic methods with exceptional system and exceptional gradualness:

- Probabilistic foundations: Bayes' rule, motion models, and observation models—corresponding to Chapter 11 of this book.
- Filter-based SLAM: EKF-SLAM, the UKF, particle filtering and FastSLAM, and the RBPF principle behind Gmapping—corresponding to Chapters 11 and 14.
- Graph-optimization SLAM: least squares, pose graphs, loop closure, and robustification—corresponding to Chapter 12.
- Mapping methods: occupancy grid mapping, laser scan matching, and map representations—corresponding to Chapter 13.

The Resource Friendliest to Self-Learners

Professor Stachniss has uploaded complete recordings of his courses over the years to YouTube, freely available, ranging from five-minute clips on a single concept to full ninety-minute lectures, with animations and blackboard derivations. SLAM learners worldwide regard them as the audiovisual resource best suited to self-study. It is worth noting in particular that Gmapping, the protagonist of Chapter 14 of this book, came out of the research of Stachniss together with Grisetti and Burgard—

watching the original authors explain it themselves after reading Chapter 14 is learning at its most authentic.

2.6 Mapping the Five Courses onto the Chapters of This Book

Before summarizing the common architecture, here is something practical: a unit-by-unit mapping of the core topics at the five schools onto the chapters of this book. The table serves two purposes—when you want to go deeper into a topic, it tells you which international course to consult; and when you take one of those online courses, it tells you which chapter of this book to refer back to.

Table 2-1 Course Units at the Five Schools and the Corresponding Chapters of This Book

[MIT VNAV: Visual Navigation and Optimization]

Image formation and camera models → Chapter 9

Optimization on manifolds → Chapter 10 (rotation representations), Chapter 12 (graph optimization)

IMU preintegration → Section 7.5

Visual-inertial odometry → Chapter 23

[CMU 16-833: Localization, Mapping, and the Back End]

Bayes and particle filtering → Chapter 11

EKF-SLAM → Section 11.6

Factor graphs and iSAM2 → Section 12.6, Chapter 27 (multi-robot)

Loop closure detection → Section 20.7, Section 22.5

[Stanford Robot Autonomy: System Integration]

Robot system architecture → Chapter 3

Perception and state estimation → Part II, Chapter 11

Path planning and control → Section 3.4, Section 29.6

[ETH / UZH: Vision and UAVs]

Feature detection and matching → Chapter 20

Multi-view geometry → Chapter 21

Visual-inertial fusion → Chapter 23

Micro aerial vehicle navigation → Chapter 26

[Bonn / Freiburg: Probabilistic Mapping]

Occupancy grid maps → Chapter 13

Scan matching and ICP → Section 13.3, Section 17.3

Gmapping and the RBPF → Chapter 14

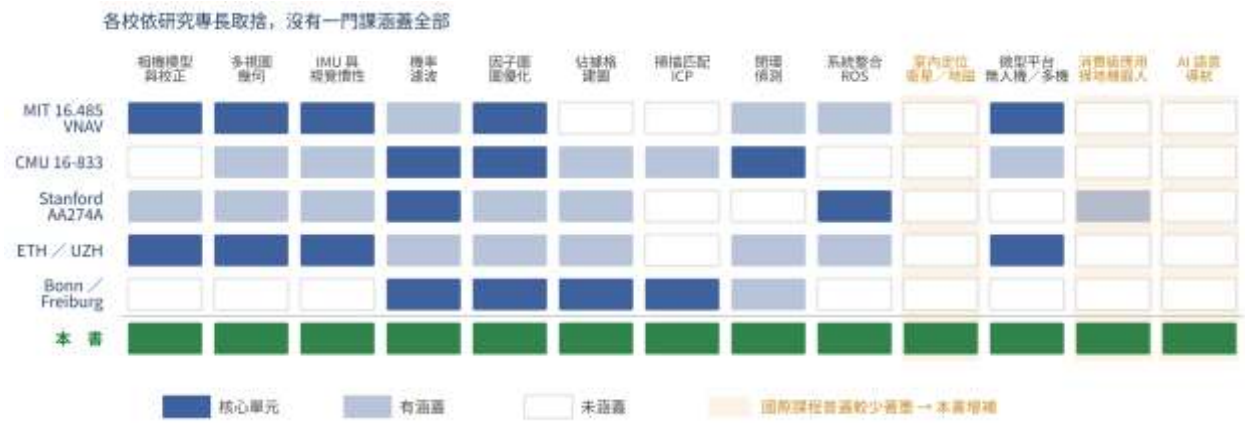
Graph-optimization SLAM → Chapter 12, Chapter 15

Looking at the table, you will notice one thing: no single school's course covers every topic. MIT leans toward visual-inertial estimation and optimization, Bonn toward probabilistic mapping, Stanford toward system integration. This is entirely reasonable—a fourteen-week course cannot exhaust an entire field, and each school

naturally selects according to its research strengths.

The editorial strategy of this book is therefore to take the strengths of each school and fill the gaps: the probabilistic mapping of Bonn forms the backbone of Parts III and IV; the visual-inertial work of MIT and Zurich is the main thread of Part VI; the factor graphs of CMU are the core of Chapter 12; the systems view of Stanford runs through Chapter 3 and Part VIII; and to these are added topics that courses in Taiwan actually need but international courses treat lightly—indoor positioning in Chapter 4, geomagnetic navigation in Chapter 6, robotic vacuums in Chapter 16, and the AI semantic navigation of Part IX.

Figure 2-1 Coverage Comparison Across the Five Courses



Drawing the mapping of Table 2-1 as a matrix makes the phenomenon described in the preceding paragraph immediately visible: five rows for the five schools, thirteen columns for the main topics of SLAM, dark cells for the core units of that course, light cells for topics covered, and blank cells for topics not covered.

Read across the rows and every school shows a clear center of gravity: the dark cells of MIT and the two Zurich institutions cluster on the left in camera models, multi-view geometry, and visual-inertial estimation; those of Bonn / Freiburg cluster in the

middle in probabilistic filtering, occupancy grid mapping, and scan matching; CMU falls on probability and factor graphs; and Stanford is the only one to treat system integration and ROS as core. This is exactly the "selection according to research strength" noted in Section 2.6.

Reading down the columns is more interesting still. Four columns are nearly blank throughout—indoor positioning together with satellite and geomagnetic navigation, consumer applications, AI semantic navigation, and miniature platforms (covered by only two schools). This does not mean these topics are unimportant; it means they either belong to industry rather than the research frontier, or evolve too quickly to have settled into a fourteen-week syllabus.

The last row is this book. It fills all thirteen columns, but you should understand that this does not make the book deeper than those courses—quite the opposite. The task of a textbook is to build a complete skeleton and a common vocabulary, while an international course provides depth in one area. The correct use of the two is complementary: use this book to cover the ground, and the courses to dig down.

2.7 The Common Architecture of SLAM Curricula: Sensors → Mathematics → Front End → Back End → Loop Closure → Capstone

Having visited five schools, if we pull the lens back we find something interesting: although the entry points differ—MIT from geometry, CMU from probability, Stanford from systems, Zurich from vision, Germany from mapping—the knowledge skeletons of these courses are strikingly consistent and can all be summarized in six stages:

Figure 2-2 The Common Architecture of SLAM Curricula

(1) Sensors: learn the data sources—camera, LiDAR, IMU—and their error characteristics

↓

(2) Mathematics: coordinate transformations, probability, filtering, optimization—the language for handling uncertainty

↓

(3) Front end: feature extraction, data association, scan matching, odometry

↓

(4) Back end: filtering or graph optimization to obtain the most consistent trajectory and map

↓

(5) Loop closure: recognize revisited places and correct globally accumulated error

↓

(6) Projects: integrate and validate on a real or simulated platform to complete a working system

The nine-part structure of this book is the result of expanding those six stages and localizing them: Part II corresponds to sensors; Part III to mathematics; Parts IV through VI realize the front end, back end, and loop closure in three kinds of system—2D LiDAR, 3D LiDAR, and vision; Parts VII and VIII correspond to the capstone; and Part IX extends to the rapidly evolving frontier of AI semantic navigation that the courses are still catching up with.

School / Course	Principal Instructor	Entry Point	Signature Feature	Corresponding Material in This Book
MIT 16.485 VNAV	Luca Carlone	Geometry and optimization	Manifold optimization, UAV projects	Chapters 10, 12, 21
CMU 16-833	Michael Kaess	Probabilistic inference	Factor graphs, iSAM / GTSAM	Chapters 11, 12
Stanford AA274A	Marco Pavone et al.	System integration	Full ROS stack, end-of-term integration	Chapters 3, 28, 29
ETH Autonomous Mobile Robots	Roland Siegwart	General survey of mobile robotics	Classic textbook, edX MOOC	Chapters 3, 7
UZH Vision Algorithms	Davide Scaramuzza	Vision and VIO	Building-block VO project, event cameras	Chapters 9, 20, 21
Bonn / Freiburg	Stachniss, Burgard	Probabilistic mapping	Fully public on YouTube, Gmapping	Chapters 11, 13, 14

Table 2-2 Overview of the Five SLAM Courses and Their Correspondence to the Chapters of This Book

Formulations and Procedures

1. Unifying the Five Courses Through the Estimation Problem

Although the five courses differ greatly in style, all of them revolve around the same central formula introduced in Chapter 1. Recall the probabilistic definition of online SLAM:

$$p(x(t), m \mid z(1:t), u(1:t))$$

The differences among the five courses amount to different strategies for attacking this one formula:

- CMU and Freiburg / Bonn concentrate on how to solve for this probability distribution: filtering methods (Chapter 11) update it recursively over time, while graph optimization (Chapter 12) converts it into a least-squares problem.
- MIT and UZH concentrate on where the observations z come from: how a camera image becomes a geometric constraint—features, epipolar geometry, and reprojection error (Chapters 20 and 21).
- Stanford concentrates on what to do with the solution: how pose and map are fed into path planning and control to constitute a complete autonomous system (Chapters 3 and 29).

2. A Procedure for Planning a Self-Study Route Through SLAM

Drawing together the resources of the five schools, this book recommends the following self-study route; you may choose your entry point according to where you start:

Figure 2-3 A Recommended Procedure for Planning Self-Study in SLAM

Starting point: do I have a background in programming and linear algebra?

├─ Not yet → begin with the ETH Autonomous Mobile Robots MOOC (concept-oriented, lowest barrier)

└─ Yes → proceed to the next step

↓

To take the vision route → UZH course videos + implement the visual odometry building blocks week by week

To take the probabilistic route → Stachniss's YouTube course + the derivations in Part III of this book

↓

Advanced depth → MIT VNAV OCW notes (manifold optimization) + CMU 16-833 (factor graphs)

↓

System integration → complete the implementation of Part VIII and the capstone of Chapter 31 using ROS

2.8 The Evolution of Open Education: From MOOCs to Embodied Intelligence Courses

The courses introduced in this chapter share one feature: their materials are public. This seems self-evident today, but it is a change of the past decade or so, and it is still unfolding. Understanding that evolution is useful when planning your own learning path.

Change in Three Stages

How knowledge is obtained has changed fundamentally over the past dozen years. The process began roughly in 2011, when professors at Stanford opened their artificial intelligence course online to everyone; before that, artificial intelligence could generally be studied only through a limited number of university courses or through books and papers, and those resources were concentrated in developed countries, so learners in emerging markets had difficulty reaching them. The appearance of massive open online courses began what is called the democratization of knowledge.

For a field like SLAM, which demands a great deal of background knowledge, this change was especially consequential. Before the 2010s, studying SLAM meant, in practice, gaining admission to one of a handful of laboratories; today any student in Taiwan with an internet connection can obtain the same lecture notes and assignments as a student at MIT. Professor Stachniss's posting of his entire course to a video platform, mentioned in Case 2-2 of this book, is a representative instance of this trend.

Stage Two: Hardware-Oriented Open Courses

Video-only courses have a limitation: students can follow the theory but have no robot on which to verify it. In response to this gap, open courses combined with physical platforms appeared. In March 2021, for example, the Duckietown Foundation released the first hardware-based massive open online course in artificial intelligence and robotics, titled "Self-Driving Cars with Duckietown," offered free on a public platform. The course has learners work hands-on so that a real robot makes its own decisions and completes broadly defined tasks, proceeding step by step from theory through implementation to deployment in simulation and on real robots; a robot platform was designed from scratch specifically to support this learning experience.

This model agrees with the design philosophy of Part VIII of this book: first get the workflow and the parameters right in simulation, then move to inexpensive physical hardware for verification. The containerized uniform environment recommended in Section 28.1 and the simulation-before-hardware path of Chapter 29 follow the same line of thought.

Stage Three: Embodied Intelligence Enters the Curriculum

In the past two years the content of such courses has changed again. Breakthroughs in large language models are being applied to robotics, and embodied intelligence allows robots to understand natural-language instructions and convert them into physical action; meanwhile organizations such as Google DeepMind and OpenAI are developing robot foundation models that learn general manipulation skills from enormous datasets, and understanding how to fine-tune such models for a specific application is becoming an essential skill.

At the same time another direction is entering the curriculum: because of the limitations of cloud connectivity, demand for on-device AI inference keeps rising, and modern courses increasingly cover model optimization techniques (quantization, pruning, knowledge distillation) together with edge AI frameworks and hardware acceleration. This is exactly the material discussed in Chapter 25 (milliwatt-class platforms) and Section 33.8 (edge deployment) of this book—these two topics were included precisely to align with this pedagogical trend.

Three Implications for How to Study

1. Simulation first: researchers have noted that simulation projects can teach roughly eighty percent of the necessary concepts at about one tenth the cost of hardware-based study; once the foundations are solid, hardware work proceeds

far more efficiently, whereas attempting complex projects too early leads to frustration. This agrees entirely with the recommendations of Part VIII.

2. Graduated challenge: the recommended learning structure advances level by level—basic motion (make the robot trace a square, a circle, a figure eight), sensor integration (add obstacle detection and avoidance), environment mapping (build a spatial map with SLAM), object manipulation (visual detection and grasping), and autonomous missions (integrate all capabilities into a complete task). In this book these five levels correspond to Chapter 3, Chapter 8, Chapters 13 through 15, Chapter 24, and Chapter 31 respectively.

3. Online courses and textbooks are complementary: online courses excel at dynamic demonstration and currency; textbooks excel at systematic organization and repeated reference. The recommended practice is to build the framework with this book, supplement intuition with online courses, and verify understanding with open-source projects—a division of labor that extends the method for reading open-source projects given in Section 28.6.

Case Studies

Case 2-1 From Course Assignment to Industry Change: The Story of GTSAM

The factor graphs and incremental smoothing taught in CMU 16-833 are not mere classroom theory. The GTSAM (Georgia Tech Smoothing and Mapping) library developed by Professors Kaess and Dellaert packages this theory into an efficient open-source C++ toolbox that is now standard equipment in robotics companies and laboratories worldwide—its traces appear in the back-end optimization of underwater

robots, self-driving cars, and AR devices alike. The lesson is this: the core assignment of a course (in 16-833, students implement a factor graph solver themselves) may be the prototype of tomorrow's industrial infrastructure. When Chapter 12 introduces g2o, Ceres, and GTSAM, you will use this tool that grew out of a classroom with your own hands.

Case 2-2 The SLAM Professor on YouTube: The Power of Open Knowledge

Beginning in 2013, Professor Stachniss published complete recordings of the Robot Mapping course from his Freiburg years and has continued to update them at Bonn, accumulating several hundred videos viewed millions of times. Countless self-learners around the world—including many graduate students and engineers in Taiwan—entered SLAM through these free videos. Likewise, MIT placed VNAV in full on OCW, UZH published all its notes and videos, and ETH launched a MOOC. This wave of openness at leading universities means that the barrier to studying SLAM has shifted from whether one can enter an elite school to whether one has a good learning map and sustained persistence. This book aims to be exactly that learning map, written in Traditional Chinese.

Case 2-3 A Taiwanese Student's Perspective: From the University Classroom to International Courses

For learners in Taiwan, the five courses make excellent material for a second, more advanced pass, but tackling the original materials directly usually meets two obstacles: the double barrier of mathematical notation and English terminology, and the absence of a locally operable experimental site. The strategy of this book is to build locally and connect internationally: first establish the concepts and the implementation firmly in

Chinese and in familiar settings (classroom mapping, campus inspection, robotic vacuums), and then point the Further Reading of each chapter to the corresponding unit of an international course. Take Gmapping in Chapter 14: first complete the mapping exercise with a robot in a university classroom, then go back and watch Stachniss explain the RBPF—most students find that the previously abstract particle filter suddenly becomes visible.

Case 2-4 Why a Textbook Is Still Needed: University Courses in the MOOC Era

This chapter has introduced a great many free online resources, which naturally raises a pointed question: if the courses of top universities can be watched for free, why enroll in a course at all? And why does this book need to exist?

The question is indeed debated seriously in education. Scholars have observed that as the digital transformation of teaching advances rapidly, universities have begun to face significant competition from massive open online courses and must therefore differentiate themselves in order to explain to students what the added value of a three- to five-year degree actually is.

In the author's view, textbooks and classrooms retain at least four kinds of value in this era that online video cannot easily replace:

- Localization: international courses are taught in English with European and American settings as examples. This book deliberately uses the professional terminology current in Taiwan (see the discussion of usage in Chapter 1), takes the university campus and the Taiwanese high school classroom as its sites, and includes topics of particular local concern (such as the behavior of geomagnetic navigation in steel-frame buildings in Chapter 6 and robotic vacuums in Chapter

16).

- Systematicity: online courses stand alone, and students must assemble the pieces themselves. The nine-part structure of this book, the "Where This Chapter Sits in the Book" section opening every chapter, and the cross-references between chapters all exist to supply that overall view.
- Referenceability: video makes it hard to look a formula or a parameter back up quickly. When you are stuck during implementation, a textbook's index, table of contents, and chapter numbering are far more efficient than rewatching a video.
- Transmission of failure experience: this is the aspect this book cultivates most deliberately. International courses usually present successful results, whereas every chapter here discusses common failures—the four failure modes of Section 14.6, the safety principles of Section 26.5, the debugging order of Section 29.7. This experience, drawn from real teaching practice, is something no video will tell you.

The position of this book is therefore clear: it does not compete with online courses but serves as their map and annotation. Students who use both—building the skeleton with this book and adding flesh with the international courses—will gain more than from any single source. That is also the real message behind the "power of open knowledge" emphasized in Case 2-2: resources are not the problem; organizing them is.

Hands-On Activity

Activity 2-1 Exploring International SLAM Course Resources and Writing a Comparison Report

Purpose: to locate and browse the public resources of the five schools firsthand, build your own map of learning resources, and practice reading English course websites.

Procedure

5. Form groups of two or three, each group claiming one school: MIT VNAV, CMU 16-833, Stanford AA274A, UZH Vision Algorithms, or the Bonn courses of Stachniss.
6. Search for and open that course's official website or its OCW / YouTube page, and find: the course title and number, the principal instructor, and the weekly syllabus of the most recent offering.
7. Choose one week whose topic title you can already understand (for example Kalman Filter or Feature Extraction), and read the first five pages of that week's notes or the first ten minutes of the video.
8. Prepare a one-page slide answering: what does this course assume students already know? What does it teach first? What do the assignments or projects look like? Which part corresponds to which chapter of this book?
9. Each group presents for three minutes, and the class together completes a comparison table of the five courses, posted in the classroom or on the course website as a reference map for the term.

Discussion Questions

6. Of the five courses, which has the lowest barrier to entry? Which demands the strongest mathematical background? On what evidence do you base your judgment?
7. If you could follow only one free resource alongside this book for self-study, which would you choose, and why?

8. (Undergraduate) Compare the project requirements of VNAV and 16-833: which form of assessment better tests whether a student has truly learned SLAM?

Extended Challenge (Undergraduate)

Download the first exercise from the UZH course website (usually on feature extraction or camera models), try to run it on your own computer, and record the environment setup problems you encounter and how you solve them—this record will prove useful when you build the SLAM development environment in Chapter 28.

Further Reading

4. MIT OpenCourseWare, 16.485 Visual Navigation for Autonomous Vehicles (VNAV), Fall 2020. (Complete lecture notes, assignments, and UAV lab guides, freely available.)
5. CMU 16-833 Robot Localization and Mapping course website (cs.cmu.edu/~kaess/teaching/16833). (Syllabi and reading lists from past years.)
6. Stanford AA274A / CS237A Principles of Robot Autonomy I course website and public lecture notes. (A full-stack course on autonomous robots.)
7. R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed., MIT Press, 2011. (The textbook assigned in the ETH course; also available as an edX MOOC.)
8. UZH Robotics and Perception Group, Vision Algorithms for Mobile Robotics course page (rpg.ifi.uzh.ch/teaching.html). (All notes and videos public.)
9. Cyrill Stachniss, Robot Mapping / SLAM lecture series, YouTube (channel: Cyrill Stachniss). (The audiovisual SLAM resource best suited to self-study.)

10. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005. (The theoretical synthesis of the Freiburg line and the principal reference for Part III of this book.)

11. D. Scaramuzza and F. Fraundorfer, "Visual Odometry: Tutorial," IEEE Robotics & Automation Magazine, 2011. (The classic introductory tutorial on visual odometry, in two parts.)

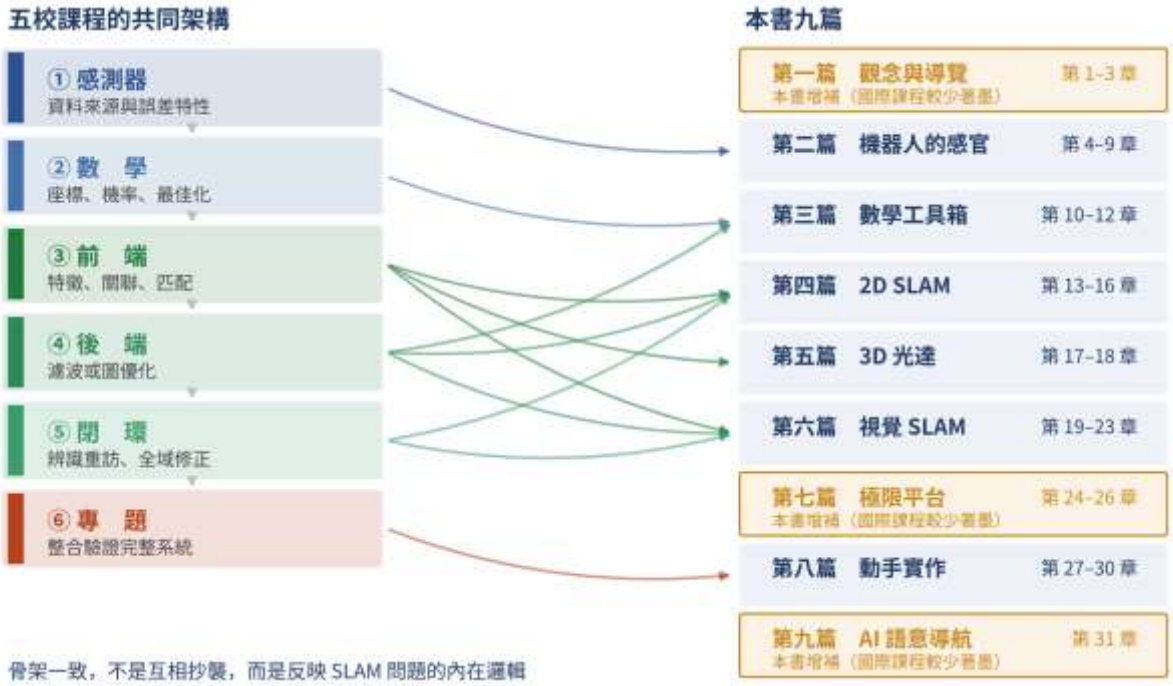
From a Tour of Courses to a System Architecture

To close this chapter, we turn the view back to the journey of this book itself.

Section 2.7 distilled the architecture common to the five courses: sensors → mathematics → front end → back end → loop closure → capstone. This architecture recurs not because the courses copy one another but because it reflects the internal logic of the SLAM problem—you must have data (sensors) before you can discuss how to process data (mathematics); you must be able to estimate the relation between adjacent instants (front end) before you can discuss global consistency (back end); loop closure is the fullest expression of the back end's power, and the capstone is the integrated test of every capability.

The nine parts of this book are that logic unfolded, with each link enlarged to its own full-length treatment:

Figure 2-4 The Common Architecture and the Nine Parts of This Book



[Material added by this book]

Part I Concepts and orientation (international courses usually enter the technology directly)

Part VII Platforms at the limit (micro-robots, UAVs, multi-robot collaboration)

Part IX AI semantic navigation (aligned with the pedagogical trend toward embodied intelligence)

In the next chapter we arrive at the first station of this architecture—not the sensors, but something above them: the complete architecture of a robot navigation system. Chapter 3 answers where SLAM sits within the whole navigation system and introduces the reactive, deliberative, and hybrid architectures, cost maps and path planning, and the differing navigation requirements of wheeled, tracked, quadruped, and aerial platforms. With that system blueprint in hand, the six sensors of Part II will each know where they belong.

Summary

8. MIT VNAV is organized around geometry and optimization on manifolds, runs a UAV project through the whole term, and has its complete materials freely available on OCW.
9. CMU 16-833 frames SLAM as real-time estimation under sensing uncertainty, concentrates on filtering, factor graphs, and the optimization back end, and is the birthplace of iSAM / GTSAM.
10. Stanford's robot autonomy course is distinguished by system integration: SLAM is taught within the complete perception–localization–planning–control stack, with ROS used throughout.
11. ETH's Autonomous Mobile Robots course (with its edX MOOC) offers the most accessible general entry point, while Scaramuzza's UZH course goes deep into vision and VIO through a building-block visual odometry project.
12. The Freiburg / Bonn line presents probabilistic mapping most systematically, and Professor Stachniss's YouTube courses are widely regarded as the free audiovisual resource best suited to self-study.
13. The five courses share one knowledge skeleton: sensors → mathematics → front end → back end → loop closure → capstone. The nine parts of this book follow that architecture and localize it to the Taiwanese campus setting.

In the next chapter we turn the lens from courses back to the robot itself and survey the complete architecture of a robot navigation system—the Sense–Think–Act loop, the taxonomy of maps and localization, and the place of SLAM within the navigation

system—preparing the ground for the tour of sensors in Part II.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 3

An Overview of Robot Navigation Systems

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 1 explained what problem SLAM solves, and Chapter 2 looked at how the world teaches the subject. But there is an easily overlooked premise: SLAM never exists in isolation—it is one link in a navigation system. This chapter draws the complete system blueprint so that you know what lies upstream of SLAM and where its output goes downstream.

This chapter closes Part I and is the key to understanding the architecture of the whole book: where in this system the sensors of Part II are mounted, which module the mathematics of Part III supports, and which components the implementation of Part VIII wires together—the answers are all in the architecture diagrams of this chapter.

Figure 3-0 How Chapter 3 Connects to the Rest of the Book

[Building on the previous chapters] The five map representations and six sensors of Chapter 1 are placed here within a complete system architecture.

[This chapter is responsible for] establishing the overall blueprint of a navigation system: how perception, decision, and action are connected, and where SLAM sits.

[Where it leads]

3.1 Sense–Think–Act → the layered thinking that runs through the book (recurring in Chapters 15, 18, 26, and 32)

3.2 Kinds of maps → occupancy grids in Chapter 13, point clouds in Chapter 17, feature maps in Chapter 20, semantic maps in Chapter 32

3.3 Kinds of localization → relocalization and the kidnapped robot problem in Chapter 11, multi-map operation in Chapter 23, AMCL in Section 29.5

3.4 Navigation system architecture → the full Nav2 implementation in Section 29.6, project integration in Chapter 31

3.5 The role of ROS → all of Part VIII (Chapters 27–30)

3.6 Platform differences → robotic vacuums in Chapter 16, quadrupeds in Chapter 18, UAVs in Chapter 26

[Prerequisites] The basic concepts of Chapter 1. This chapter still involves no

mathematics.

Learning Objectives

The previous two chapters introduced the central problem of SLAM and the teaching paths of leading universities. This chapter raises the view to survey the complete system architecture of an autonomous navigation robot, so that you hold a map of the whole system before going deep into any individual technique. After completing this chapter, you should be able to:

12. Explain the basic operating principle of a robotic system in terms of the Sense–Think–Act loop and give concrete examples of each stage.
13. Distinguish the data structures, strengths, weaknesses, and appropriate uses of the four principal map representations (occupancy grid map, feature map, point cloud, semantic map).
14. Distinguish the four modes of localization—relative localization, absolute localization, map-based localization, and relocalization—and describe the error characteristics of each.
15. Draw the architecture of a navigation system composed of the four modules SLAM, path planning, obstacle avoidance, and control, and describe the data flow among them.
16. Explain the core concepts of ROS / ROS 2 (nodes, topics, services) and the role they play in SLAM teaching and research.
17. Compare the differing navigation requirements of five platforms: the robotic vacuum, the service robot, the quadruped, the UAV, and the self-driving car.

Core Concepts

Term (Chinese)	Term (English)	Explanation
感知—決策—行動	Sense–Think–Act	The basic loop of a robotic system: sense the environment, make a decision, execute an action.
特徵地圖	Feature Map	A map that records only the positions of salient landmarks in the environment (corners, columns, markers).
相對定位	Relative Localization	Propagating pose by accumulating motion measurements (dead reckoning); error accumulates over time.
絕對定位	Absolute Localization	Measuring position directly with respect to a global reference frame (GPS, UWB); does not drift.
地圖定位	Map-based Localization	Estimating pose by matching live sensor data against an existing map, as in AMCL.
重定位	Relocalization	The ability to recover one's position within a map after becoming lost or being moved.
綁架機器人問題	Kidnapped Robot Problem	The extreme test of moving a robot abruptly to an unknown position, used to examine relocalization capability.
路徑規劃	Path Planning	Finding a feasible and optimal path from start to goal within a map.
避障	Obstacle Avoidance	Detecting and evading dynamic or temporary obstacles absent from the map.
代價地圖	Costmap	A navigation map in which obstacles are inflated by a safety margin, maintained in global and local layers.
機器人作業系統	ROS / ROS 2	A robotics software development

Term (Chinese)	Term (English)	Explanation
		framework providing inter-module communication, tools, and an ecosystem.
節點與話題	Node / Topic	The basic units of ROS: a node is a functional module, and a topic is a data channel between nodes.

3.1 Sense–Think–Act: Perception, Decision, Action

The operation of every autonomous robot—whether a robotic vacuum, a quadruped, or a self-driving car—reduces to one continually repeating loop: Sense, Think, Act. This loop executes from a few to several hundred times per second and constitutes the heartbeat by which a robot is alive.

- Sense: gather information through sensors. This includes sensing the external world (the LiDAR detects a wall ahead, the camera sees a pedestrian) and sensing the robot's own state (how far the wheels have turned, how far the body is tilted). The six major sensors of Part II all belong to this layer.
- Think: convert sensed information into action commands. This layer subdivides further: state estimation (where am I?—SLAM and localization), environmental understanding (what is around me?), task planning (where must I go?), path planning (how do I get there?), and motion planning (what is the next move?).
- Act: turn commands into actual output from motors, servos, and propellers, changing the state of the robot and of the world—which in turn changes what is sensed on the next cycle, so the loop continues.

Early robotics favored the strictly sequential Sense–Model–Plan–Act architecture, in which a complete world model had to be built before every action, making the system sluggish; in the 1980s Rodney Brooks proposed the behavior-based architecture, which skips modeling and lets perception drive behavior directly, giving fast reactions but making complex tasks difficult. Modern robots generally adopt a hybrid architecture: a fast reactive loop at the lower level handles obstacle avoidance and balance (milliseconds), while a deliberative planning layer above handles mapping and navigation (seconds)—and SLAM is a core member of that upper deliberative layer.

An everyday analogy: swerving on a bicycle to avoid a dog that darts out relies on reflex that bypasses conscious thought (reactive); planning how to ride from home to school relies on the map and the route in your head (deliberative). What is true of people is true of robots.

3.2 Kinds of Maps: Occupancy Grid, Feature Map, Point Cloud, and Semantic Map

Chapter 1 introduced the appearance of maps from the standpoint of output; this section compares them in depth from the standpoint of data structure, because the choice of map representation directly determines memory usage, algorithmic complexity, and the capabilities an application can have.

1. Occupancy Grid Map

Space is divided into a regular grid, and each cell stores an occupancy probability between 0 and 1. The advantages are a simple structure, fast queries, and natural support for path planning and obstacle avoidance; the drawbacks are that memory grows linearly with area, the resolution is fixed (with 5 cm cells a 3 cm object is invisible), and no semantics are carried. The 2D occupancy grid is the mainstream choice for indoor wheeled robots; the 3D version commonly uses an octree (such as OctoMap) for compressed storage, subdividing cells only where something exists.

2. Feature Map

This takes the opposite approach: instead of recording whether every cell is occupied, it records only the positions and descriptors of a small number of salient landmarks in the environment—wall corners, columns, AprilTags, visual feature points. The advantages are extreme compactness (a classroom may require only a few dozen

landmarks) and a natural fit with estimation theory such as EKF-SLAM; the drawbacks are that nothing is known about the space between landmarks, so the map cannot be used directly for obstacle avoidance, and that the environment must contain stable, recognizable features. The map points built by the ORB-SLAM family are essentially a visual feature map.

3. Point Cloud Map

This stores the raw three-dimensional points measured by the sensor directly, without gridding and without extracting features. The advantages are the highest fidelity and the ability to represent volumetric structure of arbitrary complexity; the drawbacks are an enormous data volume (often several gigabytes), the absence of a distinction between empty and unknown (does the absence of points mean nothing is there, or that nothing was scanned?), and the inconvenience of planning paths directly on it. In practice a point cloud often serves as raw material that is converted into a grid map or a mesh model as needed.

4. Semantic Map

Meaning is annotated on top of geometry: this region is the kitchen, that object is a chair, this line is a traversable corridor. A semantic map allows a robot to carry out tasks stated in human language and is the basis of service robots and of the AI semantic navigation of Chapter 32. It usually does not exist on its own but as a label layer overlaid on one of the three geometric maps above.

Map Type	Data Structure	Memory	Strengths	Limitations
Occupancy	Probabilistic	Medium	Convenient for	Fixed resolution;

Map Type	Data Structure	Memory	Strengths	Limitations
grid map	grid	(grows with area)	planning and obstacle avoidance; mature algorithms	no semantics
Feature map	List of landmarks	Very small	Compact; well suited to estimation theory	Cannot support obstacle avoidance; depends on features
Point cloud map	Set of three-dimensional points	Large	Highest fidelity; complete volumetric structure	Hard to plan on directly; no notion of free space
Semantic map	Geometry + labels	Depends on the underlying layer	Supports language-level tasks and human-robot interaction	Depends on the correctness of AI recognition

Table 3-1 Comparison of the Four Map Representations

3.3 Kinds of Localization: Relative, Absolute, Map-Based, and Relocalization

1. Relative Localization (Dead Reckoning)

Starting from a known origin, the current pose is propagated by accumulating the displacement of each step from wheel odometry and the IMU; this is also called dead reckoning. It is the method of sailors in the age of exploration: record heading and speed and accumulate to estimate the ship's position. Its advantages are self-sufficiency and rapid update; its fatal flaw is the drift emphasized repeatedly in Chapter 1—error only grows.

2. Absolute Localization

Position is measured directly with respect to a global coordinate frame: GPS gives latitude and longitude, UWB anchors give indoor coordinates, and a magnetic compass gives absolute heading. The advantage is bounded error that does not deteriorate over time; the drawbacks are dependence on external infrastructure (satellites, anchors), limited accuracy, and a usually low update rate.

3. Map-Based Localization

Given an existing map, the pose is inferred by matching live sensor data against the map. The classic example in the ROS ecosystem is AMCL (Adaptive Monte Carlo Localization): a large number of particles representing guessed positions are scattered over the map, and the particles whose expected laser scan best matches the actual scan are favored, gradually converging to the true pose (the principle is detailed in Chapter 11 and the implementation in Chapter 29). The reason a robotic vacuum need not remap on its second cleaning run is map-based localization.

4. Relocalization

If the robot is picked up and carried to another room (the famous kidnapped robot problem), or if visual SLAM loses track because of rapid rotation, the system must recover its position within the entire map without knowing where it is—this is relocalization. It is considerably harder than ordinary localization because there is no previous position to refer to and a global search is required. The ORB-SLAM family implements relocalization by retrieving similar images quickly with a bag-of-words model (Chapter 22); this capability shares its technical core with loop closure detection.

The four kinds of localization are not mutually exclusive but complementary layers: a practical system uses relative localization for high-rate short-term estimates, map-

based or absolute localization for drift-free long-term correction, and stands ready to invoke relocalization whenever it becomes lost—this is the sensor fusion philosophy expressed at the level of localization.

3.4 Navigation System Architecture: SLAM, Path Planning, Obstacle Avoidance, and Control

With a map and localization in hand, a robot still cannot walk to a destination. A complete navigation system consists of four modules connected by well-defined data interfaces:

Figure 3-1 Architecture and Data Flow of an Autonomous Navigation System



The left column of the figure shows the data flow through five modules connected in sequence, while the right column adds two pieces of information not easily seen at a glance in prose: the typical update rate of each module, and the actual component it corresponds to in ROS / Nav2 together with the relevant chapter of this book.

The differences in update rate deserve particular attention. Sensors output tens to hundreds of measurements per second, the controller runs fifty to one hundred times per second, and the global planner may run only once every several seconds—this is not a matter of importance but of the nature of each task: avoiding a wall must be immediate, whereas replanning an entire route can take its time. When tuning the system in Chapter 29, if the robot responds sluggishly or the path oscillates, the first thing to check is which rung of this frequency ladder has fallen behind.

The red loop arrow on the far left is the soul of the diagram: once the robot moves, the environment relative to it has changed, so it must sense again. Navigation is never a one-way pipeline but a closed loop that continually returns to itself—this is the full form of the Sense–Think–Act loop of Section 3.1.

- The SLAM / localization module is the eyes and the inner map of the whole system: without a correct pose, every module downstream works from a false premise—which is why this book devotes an entire volume to making this one module clear.
- The global planner operates on the static map and answers which general direction to take; graph search algorithms such as A* and Dijkstra are commonly used.
- The local planner operates at high rate on a local cost map covering a few meters around the robot and handles what the map does not contain: people walking by, a chair placed temporarily. The cost map inflates obstacles outward by a safety margin so that the robot does not scrape past them.

- The controller translates the geometric path into language the motors understand. For a differential-drive robot the command is usually a pair (v, ω): how fast to move forward and how fast to turn.

The concrete realization of this architecture in the ROS ecosystem is the well-known Navigation Stack (ROS 1) and Nav2 (ROS 2). Chapter 29 will have you run this entire pipeline on a physical robot.

SLAM's Upstream and Downstream: A Chart of Responsibilities

The four modules of the navigation system have been described. Here we clarify something students often confuse: where exactly does SLAM's responsibility end? What does it not do?

Figure 3-2 The Boundaries of SLAM's Responsibility, Upstream and Downstream



This figure divides SLAM's responsibility into four blocks, and from left to right they form a data chain: what upstream feeds it, what it is responsible for, what it is explicitly not responsible for, and who downstream consumes its output.

The contrast between the middle two blocks is the point of this section. Many beginners blame SLAM when the robot takes a wrong route, bumps into a chair, or

stops inaccurately, but these three belong respectively to the planner, local obstacle avoidance, and the controller. SLAM answers only two questions: where am I, and what does the environment look like.

This chart has a practical value: it lets you localize a problem quickly during debugging. Section 29.7 provides a systematic checking order whose logic follows exactly this upstream-to-downstream chain—if the robot misbehaves, first confirm that upstream (sensors, TF, timing) is sound, then look at SLAM itself, and only then check the downstream planning and control. Many students spend hours tuning planner parameters when the real problem is a mis-entered extrinsic calibration—precisely the cost of not understanding where responsibility lies.

3.5 The Role of ROS / ROS 2 in Teaching SLAM

If SLAM is the soul of navigation, ROS (Robot Operating System) is the skeleton that carries it. Despite the words "operating system" in the name, it is a robotics software framework that runs on top of Linux and solves one key problem: allowing modules written by different authors—sensor drivers, SLAM, planning, control—to be assembled like building blocks.

Three Core Concepts

- Node: an independent functional program. The LiDAR driver is a node, Gmapping is a node, the path planner is a node.
- Topic: the data channel between nodes, following the publish/subscribe model. The LiDAR node publishes the /scan topic and the SLAM node subscribes to it—neither needs to know the other, so long as they agree on the data format.

- Service and Action: the request-response pattern (querying a map) and the long-running task pattern (navigate to a goal, with progress reporting and cancellation).

Why Teaching SLAM Is Inseparable from ROS

First, a standardized interface makes algorithms hot-swappable: with the same robot and the same laser data, swapping one node lets you compare the map quality of Gmapping and Cartographer—this is the basis of the experiments designed in Part IV. Second, the rosbag recording tool captures sensor data for repeated playback, so students without a robot can still practice SLAM fully using public datasets. Third, the RViz visualization tool displays the map, pose, particle cloud, and planned path in real time, turning abstract algorithms into visible animation. Fourth, the mainstream open-source SLAM systems worldwide (Gmapping, Cartographer, RTAB-Map, and community wrappers of ORB-SLAM) all provide ROS interfaces, so standing on ROS means standing on the whole ecosystem.

From ROS 1 to ROS 2

ROS 1 was born in 2007 and reached end of official support in May 2025 with its final release, Noetic; new projects should use ROS 2 without exception. ROS 2 redesigned the underlying communication (adopting DDS) and supports real-time operation, multi-robot systems, and embedded platforms. A new version is released every year on May 23 (World Turtle Day): even years are long-term support releases (LTS, supported for five years) and odd years are regular releases. As of this writing, commonly used versions include Humble (2022, LTS), Jazzy Jalisco (2024, LTS), and the latest Lyrical Luth (2026, LTS, paired with Ubuntu 26.04). The implementation chapters of this book are based on LTS versions, and Chapter 28 explains environment

installation in detail. The good news pedagogically is that the concepts of nodes and topics introduced in this chapter carry over directly from ROS 1 to ROS 2—learn one and you know both.

Nav2's Standing Today: From Research Tool to Industry Standard

This section has described the role of ROS in teaching; here is an additional observation relevant to career planning: the navigation stack of the ROS ecosystem has evolved from a research tool into an industry standard.

As the successor to the ROS Navigation Stack, Nav2 builds on fifteen years of inheritance. It takes techniques from the autonomous vehicle field, scales them down, and redesigns and optimizes them for mobile and ground robots; it provides a complete set of functions—perception, planning, control, localization, visualization, and behavior—that compute an environment model from raw or preprocessed sensor and semantic data, plan paths dynamically, generate feasible motor commands, avoid obstacles, and organize higher-level robot behaviors.

Its degree of industrial adoption is worth noting: Nav2 is a production-grade, high-quality navigation framework trusted by more than a hundred companies worldwide; it enables mobile robots to complete user-defined application tasks in complex environments and applies to nearly every class of robot kinematics. Its capabilities are not limited to moving from point A to point B—intermediate poses can be specified, and other classes of task such as object following and complete coverage navigation can be expressed.

The last point is particularly meaningful for this book: complete coverage navigation is exactly the core task of the robotic vacuum in Chapter 16, and object

following echoes the visual navigation of Chapter 24. This shows that Nav2 is not merely a path planner but a composable task framework—Section 29.6 explains how it achieves this through the concept of behavior trees.

Study Note: The Market Value of ROS Skills

For students in Taiwan, practical competence in ROS 2 and Nav2 has several concrete values:

• A common industry language: whether the work is AGVs, service robots, agricultural robots, or inspection quadrupeds, most teams use ROS 2.

• A portfolio that accumulates: the capstone results of Chapter 31 can go straight onto a résumé and can be reproduced and verified by others.

• A bridge to international work: most of the assignments in the courses introduced in Chapter 2 also use ROS as their platform.

Part VIII of this book (Chapters 27–30) is therefore not merely a teaching unit but professional training—

the environment setup of Chapter 28, the complete navigation system of Chapter 29, and the capstone assessment of Chapter 31

together constitute a demonstrable portfolio of implementation work.

3.6 From the Robotic Vacuum to the Self-Driving Car: Navigation Requirements Across Platforms

The same navigation architecture takes on strikingly different forms on different platforms. Understanding how requirements shape a system is where the ability to design systems begins.

- Robotic vacuum: a 2D plane, low speed, a semi-static environment. The objective is not to go from A to B but to cover every square inch of floor (coverage path planning), and sensor cost is squeezed to the limit—one 2D LiDAR plus bump and cliff sensors must do everything (Chapter 16).
- Indoor service robot (food delivery, guiding): coexistence with people is the greatest challenge. It requires semantic understanding (elevator, reception desk), socially aware obstacle avoidance (not forcing a path through a crowd), and the ability to keep the map updated over long-term operation.
- Quadruped: able to climb stairs and step over obstacles, so navigation is upgraded from 2D to 2.5D / 3D—the map must contain terrain height and steppability, localization must stay stable under violent shaking, and tightly coupled LiDAR-inertial estimation (Chapter 18) becomes standard.
- UAV: full six degrees of freedom, high speed, and severe limits on payload and power. Lightweight visual-inertial solutions are preferred (Chapter 26), and latency tolerance is zero—half a second late with the pose and the aircraft has already hit the wall.
- Self-driving car: high speed, grave consequences, and large scale. It uses a high-definition map plus multiply redundant sensing (LiDAR, camera, radar, and RTK-GNSS are all indispensable), demands centimeter-level localization accuracy, and must pass automotive-grade safety validation.

Platform	Spatial Dimension	Speed	Core Difficulty	Typical Sensor Suite
Robotic vacuum	2D	Slow	Complete coverage; extreme cost limits	2D LiDAR + bump sensors + IMU
Service robot	2D + semantics	Slow	Coexistence with people; long-term map maintenance	2D LiDAR + camera + ultrasound
Quadruped	2.5D / 3D	Medium	Terrain understanding; vibration	3D LiDAR + IMU + depth camera
UAV	3D (6-DoF)	Fast	Weight and power; zero latency tolerance	Camera + IMU (VIO)
Self-driving car	2D over a large area	Very fast	Safety redundancy; centimeter accuracy	3D LiDAR + camera + radar + RTK

Table 3-2 Comparison of Navigation Requirements Across Five Platforms

The chapter sequence of this book advances along exactly this spectrum: Part IV starts from the most approachable robotic vacuum and 2D SLAM, Part V moves to quadrupeds and 3D LiDAR, and Parts VI and VII go on to visual navigation and UAVs, letting you climb the ladder of difficulty one rung at a time.

Figure 3-3 The Spectrum of Navigation Requirements Across Five Platforms



Once Table 3-2 is drawn as parallel columns, a clear spectrum emerges: from left to right, the spatial dimension rises from 2D to 3D, speed goes from slow to very fast, and the sensor suite expands from a single 2D LiDAR to LiDAR plus camera plus radar plus RTK.

Note the chapter labels along the bottom row—they are almost exactly the order of Parts IV through VII of this book. This is no coincidence: the book is deliberately arranged along this spectrum of difficulty, so that you begin with the most approachable, least expensive, and most easily verified robotic vacuum and move step by step to quadrupeds, UAVs, and multi-robot collaboration. Each step to the right raises the degrees of freedom, the speed, and the uncertainty that must be handled by one level.

Another observation worth savoring concerns the self-driving car: its spatial dimension returns to 2D (cars do not fly), yet large-scale 2D brings an entirely different set of difficulties—the map is too large to load in full, the speed leaves the algorithm only tens of milliseconds, and the consequences are grave enough to require multiple redundancy. This illustrates one thing: low dimensionality does not mean simplicity,

and what truly determines difficulty is the combination of requirements.

Formulations and Procedures

1. Error Accumulation in Dead Reckoning

Take a 2D differential-drive robot. Let the pose at time t be (x(t), y(t), θ(t)), and let linear velocity v and angular velocity ω be measured over one sampling period Δt. The dead-reckoning update equations are:

$$x(t+1) = x(t) + v \cdot \Delta t \cdot \cos\theta(t)$$

$$y(t+1) = y(t) + v \cdot \Delta t \cdot \sin\theta(t)$$

$$\theta(t+1) = \theta(t) + \omega \cdot \Delta t$$

Note that v and ω at every step carry measurement error, and that the error is transmitted to the position through the cosine and sine—particularly the angular error: a heading off by one degree puts the position off by about 17 centimeters after ten meters of straight travel, and every subsequent step builds on that skewed direction. This simple pair of equations is the mathematical explanation of why wheel odometry alone cannot support navigation, and a preview of the coordinate kinematics of Chapter 10 and the error propagation theory of Chapter 11.

2. Decomposing the Navigation Problem

The system architecture introduced in this chapter can be organized as the three questions of navigation and their corresponding modules:

Figure 3-4 The Three Questions of Navigation

Question 1: Where am I?

→ Localization / SLAM (Parts I through VI of this book)

Question 2: Where am I going?

→ Task and goal decision making (the capstones of Chapters 31 and 32)

Question 3: How do I get there?

→ Path planning + obstacle avoidance + control (the Nav2 implementation of Chapter 29)

These three questions come from a classic formulation in mobile robotics (Leonard and Durrant-Whyte): the task of this entire textbook is to answer the first question thoroughly and to equip you to connect that answer to the second and third.

3.7 The Contemporary Evolution of Navigation: From Modular Design to Foundation Models

What this chapter describes is the classical modular navigation architecture—perception, localization, planning, and control each with its own duty, connected by well-defined interfaces. This architecture is stable, interpretable, and easy to debug; it is the backbone of the overwhelming majority of deployed systems today and the object of the implementation work in Part VIII.

In the past two years, however, an entirely different line has appeared, and it is worth knowing that it exists and where it stands.

The Second Line: Navigation Foundation Models

In the traditional architecture every module is trained or designed separately; the new line attempts to accomplish navigation end to end with a single large model. Such models are called navigation foundation models or vision-language-action models.

Their range of capability is remarkable. One representative work, for example, provides a unified framework for several kinds of task—including image question answering, video question answering, and navigation; it is validated on benchmarks such as object search, tracking, and autonomous driving, and tested on a variety of real robot platforms including humanoids, quadrupeds, UAVs, and wheeled robots, indicating a possible path toward general-purpose navigation.

This property of one model across many platforms makes an interesting contrast with the discussion in Section 3.6: that section shows how greatly navigation requirements differ across platforms (wheeled robots need only 2D, quadrupeds need terrain assessment, UAVs need six degrees of freedom), while the foundation model line attempts to cover those differences with a single model. Section 24.5 and Chapter 32 return to this theme.

World Models: Letting the Robot Imagine the Future

Another emerging concept is the world model. Traditional navigation plans on the basis of the current map; a world model lets the robot predict what it will see if it moves in a given direction and choose its action accordingly. Recent research has applied vision-language world models to hierarchical planning and dual-horizon navigation prediction, and other work explores how a world model can evolve on its own in a continuous environment.

This line of thought will not be unfamiliar to readers of this book—the Bayes filter of Chapter 11 is itself a predict-update cycle, except that what is predicted is the pose rather than the image. A world model may be understood as extending the object of prediction from state to perception, a natural but powerful extension.

The Pace of Growth and a Pragmatic Judgment

The growth rate of this field deserves attention. One compilation notes that more than ninety robot foundation models were released between 2025 and 2026, spanning vision-language-action models, world models, humanoid foundation models, and policies for locomotion, navigation, and manipulation; and that submissions on vision-language-action models to top conferences grew roughly eighteenfold from 2025 to 2026.

How should a student respond to change at this pace? This book offers three recommendations:

1. Master the modular architecture first, because it is the mainstream of actual deployment and the basis for understanding new methods—a foundation model must still output a pose or waypoints, must still contend with sensor limitations, and must still avoid obstacles in the real world.
2. Understand which old problem the new method solves: the value of foundation models lies in generalization and a language interface, not in replacing geometric precision. Section 34.7 discusses their four major risks in detail, including hallucination, systematic misjudgment, latency, and safety.
3. Think in layers rather than replacement: the most pragmatic integration today has the foundation model handle high-level decisions (on the order of seconds and above) while the geometric system handles real-time control (milliseconds).

This is exactly the layered architecture proposed in Section 32.6 and a modern version of the Sense–Think–Act idea of Section 3.1.

Concept Summary: The Division of Labor Between the Two Lines

[Modular architecture] Parts I through VIII of this book

Strengths: interpretable, debuggable, well-defined computational requirements, no training data needed

Limitations: each module is optimized separately and global optimality is not guaranteed; weak semantic understanding

[Foundation model line] Chapter 32 of this book

Strengths: strong generalization, a language interface, portability across platforms

Limitations: high computational and memory demands, susceptibility to hallucination, difficulty of debugging and accountability

[Pragmatic integration] Let the foundation model understand the task at the upper layer and the modular architecture guarantee safety at the lower.

This principle of division recurs in Chapters 24, 27, and 32 of this book.

Case Studies

Case 3-1 A Day in the Life of a Food-Delivery Robot: The Whole Architecture Mobilized

Watch a food-delivery robot in a restaurant complete one errand and every concept of this chapter appears at once. Power-on: it loads the existing map and confirms with AMCL map-based localization that it is at the charging dock (map-based localization plus relocalization capability). Order received: the task system designates table 12 (the second question of navigation). Departure: the global planner computes a route on the occupancy grid map (A*), wheel odometry and the IMU supply high-rate poses along the way (relative localization), and the laser continually matches against the map to correct them (map-based localization). Midway a guest stands up and blocks the aisle: the local cost map updates instantly, the local planner traces an arc around the obstruction (obstacle avoidance), and the controller issues a new (v, ω). After delivery it returns to charge. Throughout the errand, the map built by SLAM is the foundation of everything—let the map be off by one meter and the robot will deliver the meal into a wall.

Case 3-2 The Kidnapping Test: Carrying a Robotic Vacuum into Another Room

Many students can run this classic experiment at home: let the vacuum work halfway through the living room, then pick it up and set it down in the bedroom. Early low-cost models go into complete disarray—believing themselves still in the living room, they bump repeatedly into bedroom walls. A model with relocalization capability pauses for several seconds, rotates once to rescan, matches the laser profile against the whole-home map in memory, and then teleports on the app's map to the correct bedroom position and resumes work. What happens in those few seconds is textbook solution of the kidnapped robot problem: discard the prior pose hypothesis, search globally, verify multiple hypotheses, converge. One act of picking the robot up and

putting it down is a free lecture in localization theory.

Case 3-3 From Willow Garage to Nav2: The Power of the ROS Ecosystem

Around 2010 the American company Willow Garage open-sourced the PR2 robot together with ROS and assembled the first-generation Navigation Stack—a complete combination of AMCL, cost maps, and global and local planners. Over the decade that followed, thousands of robots worldwide (including many homebuilt platforms in university laboratories in Taiwan) did not have to rewrite a navigation system but stacked their own applications on the same open-source building blocks. In the ROS 2 era, Nav2 went further by introducing a behavior tree task architecture and plug-in planners. This history illustrates what open-source ecosystems mean for robotics education: a student's energy can be concentrated on understanding principles and inventing applications rather than reinventing every wheel—which is why Part VIII of this book uses ROS 2 directly as its implementation backbone.

Case 3-4 Semantics Enters the Navigation Stack: When a Segmentation Model Meets Nav2

The modular architecture described in this chapter has been undergoing a quiet but important change in recent years: semantic information has begun to enter the core of the navigation stack.

A traditional cost map (Section 29.6) distinguishes only traversable from obstructed. Real-world traversability is far more nuanced—sidewalk and roadway are both flat ground and geometrically indistinguishable, yet semantically entirely different; grass can be driven over while a puddle is better avoided; glass does not exist for a LiDAR but is plainly visible to a camera.

Practical demonstrations have recently combined segmentation foundation models with Nav2 to perform semantic navigation outdoors by segmenting sidewalk from roadway, running on commodity-grade computing platforms. This means semantic navigation has moved from papers to deployable systems.

In the architecture of this book, this change occurs at the junction of two layers: the semantic layer (the AI vision models of Chapter 32) produces the judgment "this is sidewalk, that is roadway," and the geometric layer (the cost map of Chapter 29) converts that judgment into a traversal cost. The interface between them is simple—the semantic result becomes one layer of the cost map. This is precisely the advantage of a modular architecture: new capability can be added as a layer without rewriting the system.

There are two lessons here. First, the layered cost map design discussed in Section 29.6 is not merely an implementation detail but an architectural decision that accommodates future extension. Second, if you undertake a capstone project in semantic navigation (Chapters 24 and 31), the most pragmatic entry point is not to rewrite the navigation system but to add one semantic layer—an approach that is inexpensive, low in risk, and immediately visible in its effect.

Hands-On Activity

Activity 3-1 Dissecting a Navigation System and Designing One on Paper

Purpose: without writing code, carry out a complete dissection and specification of a navigation system with the eye of a systems designer, and build a global view.

Procedure

10. Form groups of three to four, and have each group draw one platform: robotic vacuum, food-delivery robot, campus inspection quadruped, indoor inspection UAV, or autonomous shuttle bus.
11. Requirements analysis: list the operating environment of that platform (indoor or outdoor? people present? terrain?), the speed requirement, and the severity of the consequences of failure.
12. Sensor selection: choose a combination from the six sensor classes of Table 1-1 in Chapter 1, and explain the purpose of each and the reason for rejecting the alternatives (including cost considerations).
13. Map and localization selection: choose a map representation from Table 3-1 and a localization strategy from Section 3.3, and justify both.
14. Draw the system architecture: following the model of Figure 3-1, draw the complete data flow for your platform and label the update rate of each module (which need milliseconds? which are fine at seconds?).
15. Each group presents for five minutes while the other groups play the customer and raise at least one awkward question (for example: what about glass partitions? what if a child pushes the robot away?).

Discussion Questions

9. In your platform, the failure of which module would have the most severe consequences? How should the system provide a backup for it?
10. If the budget were cut in half, which sensor would you give up first? What would the system's capability degrade to?

11. (Undergraduate) In the architecture you designed, which modules can use off-the-shelf components from ROS 2 / Nav2, and which must be developed yourselves?

Extended Challenge (Undergraduate)

Install ROS 2 and the TurtleBot3 simulator (installation steps are previewed in Chapter 28, and ample tutorials exist online), launch Nav2 in the Gazebo simulation world, observe the global path, local path, and cost map in RViz, and identify each component against Figure 3-1 of this chapter.

Further Reading

12. R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed., MIT Press, 2011. (The principal reference for the systems view of this chapter; its Chapter 1 opens with the three questions of navigation.)

13. J. J. Leonard and H. F. Durrant-Whyte, "Mobile Robot Localization by Tracking Geometric Beacons," IEEE Transactions on Robotics and Automation, 1991. (One of the classic sources of the "where am I" formulation.)

14. R. Brooks, "A Robust Layered Control System for a Mobile Robot," IEEE Journal of Robotics and Automation, 1986. (The founding work of behavior-based architecture; essential for understanding how architectures evolved.)

15. S. Macenski et al., "The Marathon 2: A Navigation System," IROS 2020. (The official paper on the Nav2 navigation system.)

16. ROS 2 official documentation (docs.ros.org) and Nav2 official documentation (docs.nav2.org). (Browsing their conceptual introduction pages before the implementation chapters is recommended.)

17. S. Macenski et al., "Robot Operating System 2: Design, Architecture, and Uses in the Wild," Science Robotics, 2022. (The authoritative introduction to the design philosophy and industrial use of ROS 2.)

18. A. Hornung et al., "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees," Autonomous Robots, 2013. (The representative work on 3D occupancy grid mapping.)

From System Blueprint to Sensors: Closing Part I

Part I ends here. The division of labor among its three chapters is as follows: Chapter 1 established the problem (what SLAM must solve), Chapter 2 provided the learning map (how the world teaches it and how this book is organized), and this chapter drew the system blueprint (where SLAM sits in the whole).

These three chapters deliberately contain no mathematics and no code, the purpose being to establish a complete conceptual framework first. The author's teaching experience is that students who enter formulas and code too early tend to miss the forest for the trees, producing a system that runs but being unable to say why it is designed as it is. With this framework in place, the technical details that follow have somewhere to attach.

Figure 3-5 From Part I to Part II

[The Three Frameworks Established in Part I]

Problem framework → localization and mapping depend on each other and must be solved together (Chapter 1)

Learning framework → sensors → mathematics → front end → back end → loop closure → capstone (Chapter 2)

System framework → perception → localization and mapping → planning → control (this chapter)

[The Questions Part II Will Answer]

With what does a robot see the world? What can each sensor do, and when does it fail?

Chapter 4 Indoor positioning → Wi-Fi, UWB, Bluetooth, visual markers

Chapter 5 Satellite navigation → GNSS principles, urban canyons, fusion with inertial sensing

Chapter 6 Geomagnetic navigation → electronic compass, magnetic interference, geomagnetic fingerprints

Chapter 7 IMU → accelerometer, gyroscope, drift, attitude filtering

Chapter 8 LiDAR → time of flight, 2D and 3D, specifications and blind zones

Chapter 9 Camera → pinhole model, calibration, monocular, stereo, and RGB-D

[The Common Structure of Part II] Every chapter unfolds as principle → specifications → failure scenarios → complementarity → implementation.

Pay particular attention to the failure scenarios discussed in each chapter. Chapter 4 treats the fluctuation of Wi-Fi signals, Chapter 5 multipath in urban canyons, Chapter 6 magnetic distortion in steel-frame buildings, Chapter 7 the rate of IMU drift, Chapter

8 LiDAR's blindness to glass, and Chapter 9 the camera's curse of the blank white wall.

These are not negative asides but the basis of engineering judgment. By Section 30.7 you will watch LiDAR and camera each break down at the same site with your own eyes—and looking back then at every chapter of Part II, you will understand why this book stresses limitations from the very beginning.

Summary

14. An autonomous robot operates on the Sense–Think–Act loop; modern systems adopt a hybrid architecture in which a low-level reactive loop handles obstacle avoidance and balance while an upper deliberative planning layer carries SLAM and navigation.
15. The four map representations each have their domain: the occupancy grid favors planning and obstacle avoidance, the feature map is compact and well suited to estimation, the point cloud has the highest fidelity, and the semantic map supports language-level tasks; practical systems commonly mix them.
16. Localization divides into relative localization (high-rate but drifting), absolute localization (drift-free but limited in accuracy), map-based localization (matching against an existing map), and relocalization (global recovery after becoming lost); the four are complementary and together constitute a complete localization capability.
17. A navigation system strings four modules into a data flow—SLAM / localization, global planning, local obstacle avoidance, and motion control; SLAM is the precondition for every module downstream.

18. ROS / ROS 2 connects modules through the standardized interfaces of nodes, topics, and services, and together with the rosbag and RViz tools has become the de facto standard for teaching and research in SLAM; ROS 1 reached end of support in 2025, and the implementation work in this book is based on ROS 2 LTS releases.
19. From the robotic vacuum to the self-driving car, navigation requirements rise step by step with spatial dimension, speed, and safety demands, and the sensor suite and system design change accordingly; the chapters of this book are arranged along exactly this spectrum of difficulty.

With this, Part I, "Introduction to SLAM and Autonomous Navigation," comes to a close—you now know what problem SLAM is, how leading universities teach it, and where it sits within the robot as a whole. From the next chapter we enter Part II and take apart the robot's senses one by one, beginning with a question that seems paradoxical: why are more than thirty GPS satellites overhead of no help indoors?